



Efficient techniques for retrieving top-K frequent itemsets

Konstantinos Malliaridis ^{ID}*, Stefanos Ougiaroglou ^{ID}

Information and Electronic Engineering, International Hellenic University, Alexander Campus, Thessaloniki, 57400, Central Macedonia, Greece

ARTICLE INFO

Keywords:

HTK-Miner

HTK-negFIN

Q-Heap

Top-*k* frequent itemsets

ABSTRACT

Frequent itemset mining is a core data mining task aimed at uncovering recurrent patterns within transactional databases. Traditional methods rely on a minimum support threshold, which is often difficult to determine. Top-*k* mining offers a pragmatic alternative by retrieving the *k* most frequent itemsets. We propose two novel algorithms: HTK-Miner and HTK-negFIN. HTK-Miner, based on equivalence class theory and breadth-first search, utilizes vertical structures with four operational modes (TS, BSN, DTS, and DBSN). Its key innovation, the Quick Heap (Q-Heap), dynamically raises the support threshold to enable early pruning and accelerated identification. Furthermore, HTK-Miner requires a single database scan and employs compressed representations to reduce execution time and memory usage. HTK-negFIN adapts the pattern-growth paradigm by extending the efficient negFIN algorithm to the Top-*k* framework, integrating the Q-Heap and shared optimizations to achieve high performance. Experiments on diverse benchmark datasets demonstrate that our proposed algorithms consistently outperform state-of-the-art methods in both runtime and memory efficiency. These results highlight HTK-Miner and HTK-negFIN as scalable, effective solutions for Top-*k* frequent itemset mining.

1. Introduction

Frequent pattern mining serves as a fundamental operation in data mining, enabling critical tasks including association rule mining, correlation analysis, sequential pattern discovery, and applications in classification and clustering (Pei et al., 2007). A pattern, also called a frequent itemset in transaction databases, is a subset of items that frequently co-occur. Throughout this paper, we will use the term frequent itemset to refer to such patterns for consistency. Frequent itemset mining was first introduced by Agrawal et al. (1993) for market basket analysis, aiming to address the problem of mining association rules. Since then, a wide range of algorithms has been proposed. One of the key reasons for the sustained interest in frequent itemset mining lies in its significant computational complexity. Even for moderately sized datasets, the search space can become enormous and grows exponentially in dense datasets making the task both challenging and resource intensive (Agrawal et al., 2014).

Most frequent itemset mining (FIM) algorithms require a minimum support threshold as an input parameter, which specifies the minimum number of transactions in which an itemset must appear within a given dataset. This parameter can be defined either as an absolute value (i.e., the exact number of transactions) or as a relative value—the ratio of transactions containing the itemset to the total number of transactions.

Although highly efficient algorithms have been proposed in this category, determining the appropriate minimum support value can be a

challenging task. A very low threshold may result in an overwhelming number of frequent itemsets and high computational cost, while a high threshold may yield very few or even no frequent itemsets, hindering user exploration and analysis.

To address this issue, algorithms were later proposed that take as input the rank number *k*, indicating the position of the most frequent itemsets to be discovered. This makes it easier to retrieve top-rank frequent itemsets, without needing to guess a minimum support value that would produce a similar result. However, the Top-*k* rank approach is not always the most effective parameter, as it is often more practical for a researcher to request the first *k* most frequent itemsets directly, rather than relying on their ranks.

In terms of operation, FIM algorithms are generally divided into two main categories: candidate generation and pattern growth. Candidate generation algorithms are mainly based on the anti-monotone property known as the Apriori principle, first introduced in Agrawal et al. (1993). Depending on how they process the data, they are further classified into those that scan the dataset transaction-by-transaction (horizontal representation) and those that construct a vertical representation, where each itemset is associated with a list of transactions (tidset) in which it appears. Pattern growth algorithms, on the other hand, utilize specialized tree structures to store the initial data in a compressed format, forming chains of frequent itemsets. These structures are then traversed recursively to create tree projections or specialized list-based representations, which are used to uncover frequent itemsets along with their support.

* Corresponding author.

E-mail addresses: konsmall@ihu.gr (K. Malliaridis), stoug@ihu.gr (S. Ougiaroglou).

The first algorithm proposed in this category was FP-Growth (Han et al., 2000), followed by many others that adopt a minimum support threshold. Representative algorithms include Pei et al. (2007), PREPOST (Deng et al., 2012), FIN (Deng & Lv, 2014), DFIN (Deng, 2016) and negFIN (Aryabarzan et al., 2018). Moreover, several algorithms from the pattern growth category have been implemented in a top- k rank manner, such as NTK (Deng, 2014), CRM (Pyun & Yun, 2014), iNTK (Huynh-Thi-Le et al., 2015), and BTK (Dam et al., 2016).

FIM algorithms can also be categorized based on their search strategy into breadth-first and depth-first approaches. Breadth-first algorithms, such as Apriori, explore the search space level by level, generating and evaluating all candidate itemsets of a given items length before moving to the next level. In contrast, depth-first algorithms, such as FP-Growth (Han et al., 2000) and Eclat (Zaki, 2000), follow each branch of the search tree as deep as possible before backtracking, often resulting in faster execution and reduced memory usage.

To provide a thorough investigation into Top- k mining efficiency, this work proposes two distinct algorithms representing the dominant FIM paradigms: HTK-Miner, a candidate-generation approach utilizing vertical data representation and equivalence class theory, and HTK-negFIN, a pattern-growth approach leveraging the specialized NegN-odeset structure. The dual development of these algorithms is motivated by the dynamic nature of the minimum support threshold in Top- k mining; by implementing both, we aim to evaluate which paradigm offers superior pruning capabilities and stability as the threshold increases. Both algorithms incorporate a lightweight Q-Heap structure and novel acceleration techniques designed to rapidly raise the support threshold and minimize search space exploration. Finally, we provide a comprehensive comparative analysis against state-of-the-art methods, such as BTK and negFIN, establishing clear guidelines on the optimal mining strategy based on dataset density and scale.

The remainder of this paper is organized as follows: Section 2 reviews related work and the background in the field of frequent itemset mining. Section 3 provides the definition of the problem, together with the necessary preliminaries. Section 4 presents the HTK-Miner algorithm, detailing its core structures and implementation to efficiently discover Top- k frequent itemsets. Section 5 presents HTK-negFIN, a Top- k mining approach based on negFIN. Section 6 presents the complexity analysis of the mining process. Section 7 reports the experimental results. Finally, Section 8 concludes the paper and outlines directions for future research.

2. Related work and background

The goal of FIM is to discover item combinations that appear together frequently in a transaction dataset. Due to its wide range of applications, including bioinformatics, text mining, product recommendation, e-learning, and web click stream analysis, FIM has become a prominent research area (Fournier-Viger et al., 2017). In addition, Top- k frequent itemset mining provides essential underlying support for modern AI applications, including intelligent recommendation and behavioral pattern discovery (Guo et al., 2025). To manage the exponential number of possible combinations, algorithms use pruning techniques to eliminate candidate itemsets, improving efficiency and scalability. Since FIM initial proposal in Agrawal et al. (1993), hundreds of algorithms have been developed, extending its scope in various dimensions: from scalable data mining techniques to handling diverse data types, advanced mining tasks such as high-utility mining and novel applications (Fournier-Viger et al., 2014; Han et al., 2007). The core idea to make mining efficient is that the user should provide the input data and a minimum support threshold ($minSup$). Itemsets with support greater than or equal to the $minSup$ are characterized as frequent.

2.1. Minimum support threshold based algorithms

This is the general category of FIM algorithms. They can generally be categorized into two main classes: candidate generation and pattern growth.

Candidate generation algorithms are based on the anti-monotone property called the Apriori principle first proposed in Agrawal et al. (1993). According to this, if a k -itemset is frequent, then all its subsets are also frequent and if a k -itemset is infrequent, then all its supersets are consequently infrequent. The first algorithm of this class, named Apriori (Agrawal et al., 1993), uses a level-wise, breadth-first search strategy to iteratively generate candidate itemsets and prune those that violate the Apriori property. It requires that the data in its horizontal form be traversed as many times as the number of items in the largest frequent itemset to be discovered.

Dozens of improvements to Apriori followed and numerous ways to traverse the data as well as many structures have been proposed for condensed data representation. In Zaki (2000), the Eclat algorithm is described, which stores the initial data in a vertical representation called a vertical tid-list, where each itemset is associated with a list of transactions in which it occurs. This technique is efficient, as frequent itemsets and their support can be computed through direct tid-list intersections without requiring multiple scans of the data. In Zaki and Gouda (2003), diffsets are introduced that only track differences in transaction IDs (tids) of a candidate itemset compared to its frequent itemsets. Diffsets reduce the memory required to store intermediate results and, when incorporated into previous vertical mining methods, significantly improve performance in dense datasets. In Dong and Han (2007), a data representation structure called BitTable is described, which stores a set of integers where each bit represents an item. Furthermore, a technique for reducing the search space, named subsume, was proposed in Song et al. (2008). By computing the subsume index, itemsets that co-occur with a representative item can be quickly identified using a breadth-first search in a single pass accelerating the mining process.

Pattern-growth algorithms do not generate candidate itemsets. The core idea is to store the initial database in compressed form within a tree-like structure, reducing memory consumption, and to employ an efficient tree-based mining method to discover frequent itemsets. The first algorithm proposed in this category is FP-Growth (Han et al., 2000), which uses several key techniques: compressing the initial database into a condensed data structure called FP-tree, using a pattern fragment growth method to avoid the costly generation of a large number of candidate sets, and applying a divide-and-conquer approach to break down the mining task into smaller and more manageable sub-tasks. FP-Growth is still considered a state-of-the-art algorithm today.

Numerous improvements have followed. In Grahne and Zhu (2005), the FP-array technique was introduced, which significantly reduces the need to traverse FP-trees, leading to improved performance. A new structure called H-struct was proposed in Pei et al. (2007) as a replacement for the FP-tree. The algorithm utilizing this structure, named H-mine, demonstrates greater efficiency in terms of memory consumption, particularly for sparse datasets, compared to its predecessors. The N-lists derived from a PPC-tree structure represent another condensed vertical database representation, proposed in Deng et al. (2012) and utilized in the PREPOST algorithm. Transactions with common prefixes share the same nodes in the PPC-tree, and support counting is achieved through the direct intersection of N-lists. PREPOST introduces a technique to reduce the search space by exploiting the single-path property of N-lists, where common frequent patterns are shared among itemsets. An improved version, named PREPOST+, was introduced in Deng and Lv (2015), which also uses N-lists to represent itemsets. It discovers frequent itemsets by employing a set-enumeration search tree combined with an efficient pruning strategy called Children-Parent Equivalence, significantly reducing the search space. The FIN algorithm (Deng & Lv, 2014) utilizes N-odesets instead of N-lists, requiring storage of only one of either the pre-order or post-order of each node. This results in lower

memory consumption and improved efficiency. DFIN (Deng, 2016) is a variation of FIN that uses DiffNodesets instead of Nodesets. Finally, in Aryabarzan et al. (2018), another algorithm named NegFIN was introduced, which relies on a novel variation of Nodeset called NegNodeset. NegNodeset employs a unique encoding model for nodes in a prefix tree based on the bitmap representation of sets. FIN, DFIN and negFIN are considered state-of-the-art algorithms for frequent itemset mining using the *minSup* threshold.

However, determining the appropriate minimum support threshold *minSup* can be a challenging task. Inability to find the right *minSup* can render the mining process ineffective. If the threshold is set too low, too many itemsets will be extracted, disproportionately burdening system resources. On the other hand, choosing a higher threshold value may result in the extraction of excessively few itemsets (Ashraf et al., 2016; Dam et al., 2016; Fang & Deng, 2008). Additionally, in many datasets, small changes to *minSup* can lead to significant fluctuations in the number of frequent itemsets discovered. For example, changing the threshold from 0.983 to 0.922 in the connect dataset (see the FIMI repository (Goethals, 2004)) can result in the extraction of 10,000 frequent itemsets (FI) instead of 100.

To address this issue, two revolutionary algorithms were proposed that did not rely on specifying the threshold *minSup*, but instead focused on the number *k* of closed frequent itemsets (Han et al., 2002; Wang et al., 2005). Both approaches need a parameter *min-l*, which is the minimal length of each itemset and it is not easy to define before the mining procedure. Still, these algorithms paved the way for subsequent, more efficient methods based on the top-*k* rank and top-*k* FP approaches which are discussed in the following sections.

2.2. Top-*k* rank frequent itemsets mining algorithms

Algorithms belonging to the category top-*k* rank FIM require the rank number *k* at which the frequent itemsets are located. Determining the rank number *k* of the itemsets based on their absolute support value is clearly easier than specifying the minimum support threshold, which varies each time depending on the input data. For example, it is easier to retrieve the top-10 ranked frequent itemsets than to determine an arbitrary support value that would retrieve the same number of frequent itemsets. The task of mining frequent itemsets of highest rank *k* involves identifying all itemsets with a rank - based on the support value - of *k* or lower. One advantage of algorithms in this category is that the rank can serve as the key in the representation structures of FI, simplifying and making their operation more efficient. Taking into account the sample dataset *D* shown in Table 2, the top-4 ranking itemsets extracted from *D* are shown in Table 3.

To our knowledge, the FAE algorithm (Deng & Fang, 2007) is the first of this kind. The algorithm employs a breadth-first strategy (level-wise) which explores the lattice level by level. Using frequent items of the current level *l* to generate candidate itemsets for the next level *l* + 1. Its major drawback is the candidate's frequent itemset support calculation, which demands many I/O scans of the initial data.

A year later, VTK was introduced by Fang and Deng (2008). VTK employs a vertical representation strategy, eliminating the need for multiple scans of the initial data. The support value of a candidate FI is computed by intersecting the frequent itemsets from the previous level.

Continuing, a new and highly promising algorithm was proposed and named NTK to extract frequent top-*k* rank itemsets (Deng, 2014). Initially, a sophisticated tree structure is constructed called PPC-Tree, which shares some similarities with the FP-Tree structure introduced in Han et al. (2000). Each node in the PPC-Tree consists of five fields: the item's name, its absolute support, a list of child nodes, and two pointers named pre-order and post-order, respectively. Using the PPC-Tree, specialized lists are created for 1-item itemsets, called Node-Lists. From these lists and using a level-wise approach, candidate frequent itemsets are generated. Shortly afterward, a significantly improved version of NTK, named iNTK, was introduced (Huynh-Thi-Le et al., 2015). This ver-

Table 1

k values for retrieving 1000 and 10,000 frequent itemsets using Top-*k* frequent itemset mining algorithms and Top-*k* rank frequent itemset mining algorithms.

k values	kosarak.dat		chess.dat		mushrooms.dat	
Top- <i>k</i> FIs	1000	10,000	1000	10,000	1000	10,000
Top- <i>k</i> Rank FIs	917	3481	247	556	164	368

sion incorporated the subsume concept presented in Song et al. (2008). Both algorithms share the disadvantage of requiring two scans of the initial dataset.

In Ashraf et al. (2016), the Top-*k* Miner algorithm was introduced, which initiated the top-*k* frequent itemset discovery process by scanning the given dataset only once. During this process, the algorithm generates a set of all the frequent itemsets of 2 elements and, in an iterative manner, uses them to build a tree structure called CIS-Tree. By employing a specialized join procedure named join-FI, the algorithm generates all possible candidate itemsets and maintains a Top-*k* Rank list to store them.

An improved version of the iNTK algorithm was introduced in Dam et al. (2016). The new state-of-the-art algorithm, named BTK, employs a structure called TB-tree to represent the initial database, which is quite similar to the PPC-tree discussed earlier. Unlike the PPC-tree in NTK and iNTK, which requires two traversals, the TB-tree needs to be traversed only once during the mining process. Furthermore, BTK implements a more effective strategy to incrementally raise the minimum support threshold based on the support of candidate itemsets and incorporates the subsume concept (Song et al., 2008). The algorithm outperformed iNTK in both run-time and memory consumption.

Continuing further, three methods for improving the top rank of the frequent itemset *k* were proposed in Abdelaal et al. (2021). These methods are based on modifications of three state-of-the-art algorithms for general minimum support threshold mining: FIN (Deng & Lv, 2014), PREPOST (Deng et al., 2012), and PREPOST+ (Deng & Lv, 2015). The new algorithms are named TK_FIN, TK_PREPOST, and TK_PREPOST+, respectively. Among these, TK_FIN is claimed to achieve the best performance in terms of both runtime and memory consumption, outperforming BTK with a 90 percent improvement in runtime efficiency.

In Iqbal et al. (2021), the TKFIM algorithm is described, which is based on the principle of equivalence classes to discover Top-*k* Identical Frequent Itemsets (IFIs), synonymous with top-*k* rank frequent itemsets. This algorithm follows a breadth-first approach and employs a pruning technique that involves progressively raising the minimum support threshold during its execution. The algorithm is open-source, which is highly significant, as most algorithm creators today do not provide open-source access to their code, hindering researchers' ability to build upon existing knowledge. However, it has some drawbacks: it fails to discover all Top-*k* IFIs, as it misses some that consist of shared itemsets; it cannot handle datasets with significant variations in transaction sizes.

While specifying a top-*k* rank value is significantly more intuitive than determining an elusive minimum support threshold, we argue that the top-*k* rank approach is often suboptimal. As demonstrated in Table 1, results between rank-based and itemset-based mining frequently diverge. For instance, in the *mushroom* dataset (Goethals, 2003), many itemsets share identical support values (ties). To retrieve exactly 1000 distinct frequent itemsets, a user would have to set *k* = 164 in a rank-based algorithm. This discrepancy underscores that top-*k* itemset mining provides more direct and predictable control over the output size than rank-based alternatives.

2.3. Top-*k* frequent itemsets based algorithms

Algorithms belonging to the category *k* Most frequent itemsets (FI) require, in addition to the input data, only the number *k* of the desired most frequent itemsets as input. Although this specific method of

Table 2
sample dataset D .

Tid	Items
1	e b f
2	e d b
3	c a e d
4	b a e
5	d c b e
6	g e d
7	a b

Table 3
Top-4 ranking itemsets extracted from D .

Rank	Itemsets	Sup
1	{e}	6
2	{b}	5
3	{d}, {ed}, {eb}	4
4	{a}	3

Table 4
Top-6 itemsets extracted from D .

#	Itemset	Sup
1	{e}	6
2	{b}	5
3	{d}	4
4	{ed}	4
5	{eb}	4
6	{a}	3

retrieving FIs seems the most useful, as it is more convenient for the user to specify the number of itemsets they desire, there are relatively few implementations of such algorithms. We believe that this is due to the complexity of the structure used to store frequent itemsets and their support. In top- k Rank FI algorithms, the structure is a dictionary where the key is the support of the frequent itemsets. Taking into account the sample dataset D shown in Table 2, the top- k rank FIs are shown in Table 3, while the top-6 frequent itemsets extracted from D are shown in Table 4.

To our knowledge, the first top- k FIM algorithms introduced are Itemset-Loop and Itemset-iLoop, proposed by Wai-chee Fu et al. (2000). In addition to the threshold k , they require the maximum number M of items that each set of items (itemset) will contain, that is, M -itemsets. These algorithms are based on the Apriori principle using the horizontal data representation. However, they are not efficient as they require multiple scans of the original dataset.

Some years later, the BOMO algorithm was presented in Yin-Ling and Wai-Chee (2004). The algorithm requires the parameter k , which determines the frequent top- k itemsets and the maximum number N of elements that the extracted itemsets will contain (N -itemsets). The algorithm was considered state-of-the-art for several years and is based on a variation of the FP-Growth algorithm (Han et al., 2000). However, since minimum support is initially set to 0, a complete FP-Tree must be constructed, including all items from the original database. This makes the algorithm less efficient, especially for datasets that do not consist of commonly frequent itemsets in transactions.

In Metwally et al. (2005), the Space Saving algorithm is described, which in fact identifies the top- k frequent itemsets. However, it is designed for streams of data (e.g. monitoring click streams (Gündüz & Özsu, 2003)), which require different handling, and therefore cannot be appropriately used for transactional data such as Market Basket Lists data (MBL-type data Malliaridis et al., 2024).

The next promising algorithms for top- k itemset mining are MTK and MTK Close (for closed frequent itemsets), introduced in Chuang et al. (2008). These are level-wise, breadth-first search algorithms that employ a δ -stair search approach. This method allocates memory to con-

currently generate candidates with consecutive itemset lengths, reducing the number of database scans needed to retrieve the top- k FPs while maintaining memory efficiency. However, they still need to undergo costly multiple scans, affecting their overall efficiency.

In Pietracaprina et al. (2010), they used sample data and defined an approximation of frequent itemsets by creating a group of itemsets that includes “all very frequent itemsets” or excludes “all very infrequent itemsets,” while estimating a bounded error in their results. However, achieving this requires additional parameters (e.g., ϵ -approximation, δ -probability), which are often difficult to determine. Furthermore, their experimental results claim to capture 95% of the frequent top- k itemsets. Nevertheless, their approach introduces a significant number of false positives and does not accurately identify the exact top- k frequent itemsets.

Continuing, two algorithms were presented in Pyun and Yun (2014), named CRM (Combination Reduction Method) and CRMN (Combination Reduction Method for the N -itemset). The efficiency of these algorithms relies on compressing the items found in a single path within the FP-Tree-like structure they generate. To achieve greater compression and reduce the operations of the combinations of itemsets related to single paths, they define the concept of a **strict closed itemset**, which is a subset of frequent closed itemsets. According to Pyun and Yun (2014), “A set of successive items becomes a strict closed itemset if the items have the same support and the support is equal to the value of the first item in any single path.” The CRM algorithm is claimed to outperform both the BOMO (Yin-Ling & Wai-Chee, 2004) and MTK (Chuang et al., 2008) algorithms mentioned earlier while simultaneously requiring significantly less memory. However, it still requires storing the entire—or nearly the entire—database in an FP-Tree-like structure, not to mention that it also requires two passes over the dataset.

Recently, a top- k frequent itemset mining algorithm (FIM), named PTF, was proposed by Wan and Han (2024). It is claimed to be capable of handling massive datasets on the scale of 10^9 transactions, which cannot be fully represented in memory. The core idea revolves around partitioning the original dataset so that frequent itemsets can ultimately be discovered from small partitions without needing to access the remainder of the data. Vertical mining is performed on the processed partitions of the vertical representation, where itemsets are generated using the high-support-first principle to accelerate in-memory computation, similar to the approach employed by HTK-Miner. Additionally, the algorithm employs a hybrid vertical storage mode, adaptively representing the vertical data in one of three formats: tid-list, diff-list, or bit vector. However, PTF mandates a predefined ordering of items within each transaction (e.g., lexicographical order) as a prerequisite for data partitioning, thereby constraining the algorithm’s generality.

To date, specialized algorithms such as Q-Viper for quantitative FIM (Czubryt et al., 2022), GMiner++ for GPU-accelerated FIM (Chon & Kim, 2024), and DP-PartFIM for privacy-preserving FIM have been proposed (Liu et al., 2024). However, these represent domain-specific solutions that do not address the broader challenges of generalized FIM and dynamic threshold management.

2.4. Research motivation

While the transition from traditional FIM to Top- k mining has been extensively studied, most existing literature focuses on adapting a single mining paradigm—either candidate generation or pattern growth—to handle the k constraint. However, the performance dynamics of these two fundamental paradigms change significantly when the minimum support threshold is not a static input but a dynamic variable that increases during execution. There is a lack of comprehensive studies that directly compare how these distinct strategies manage the trade-off between memory consumption and computational overhead under the pressure of a shifting threshold.

The necessity of implementing both HTK-Miner and HTK-negFIN arises from the inherent differences in how they explore the search

space. HTK-Miner, representing the candidate generation paradigm, relies on a breadth-first search and vertical data formats, aiming to maximize pruning by raising the *minSup* as early as possible in each level of the search tree. In contrast, HTK-negFIN, representing the pattern growth paradigm, utilizes the efficient NegNodeset structure to avoid the combinatorial explosion of candidates. By implementing both, we can investigate whether the 'test-before-expand' logic of candidate generation or the 'structure-driven' logic of pattern growth is more resilient to the challenges of Top-*k* mining.

Furthermore, proposing two algorithms allows for a more nuanced understanding of the applicable scenarios for Top-*k* mining. In real-world transactional datasets, density and item distribution vary wildly. Our dual approach serves as a benchmark to determine if a single, universally dominant strategy exists or if the choice of algorithm should be dictated by specific dataset characteristics. This investigation provides a clear roadmap for selecting the optimal mining strategy: using HTK-Miner when candidate pruning is most effective, or HTK-negFIN when the efficiency of nodeset-based pattern growth is required.

3. Preliminaries

Let $I = \{i_1, i_2, \dots, i_n\}$ be a finite set of distinct elements. Let data D be a set of transactions $\{t_1, t_2, \dots, t_m\}$, where each transaction $t \in D$ is a list of items $\{i_1, i_2, \dots, i_l\} \subseteq I$.

Definition 1. Itemset / pattern: A set of items $x \subseteq I$ is called **itemset** or **pattern**. An itemset consists of l items, which is called **l-itemset**.

The support of an itemset x , denoted as $\text{sup}(x)$ or $|x|$, is defined as the number of transactions in which x appears as a subset:

$$\text{sup}(x) = |x| = |\{t \in D \mid x \subseteq t\}|$$

Definition 2. minimum support threshold: Let $\text{minSup}(0 \leq \text{minSup} \leq m)$, where m the number of transactions in D , be a user-defined parameter called **minimum support threshold**. an itemset p is said to be **frequent** if and only if $\text{sup}(p) \geq \text{minSup}$.

Definition 3. top-*k* frequent itemsets: Given a transactional dataset D and a parameter k , **top-*k* frequent itemsets** are the $K + n$ ($n \geq 0$) most frequent itemsets FP_k where $|FP_k| = K + n$ and $\forall i \in FP_k, \forall j \notin FP_k, \text{sup}(i) > \text{sup}(j)$ where n is the number of frequent itemsets $|FP_n|$ and $\forall z \in FP_n, \text{sup}(z) = \text{minSup}$.

For example, if the parameter k is set to 1000 and the thousandth top-*k* frequent itemset, along with the next four others ($n = 4$), has the **exact same support**, then 1004 ($K + n$) frequent itemsets will be returned.

Lemma 1. Downward closure property: If an itemset X is frequent, then all subsets of X are also frequent.

Lemma 2. Anti-monotone property: Given an itemset X then for any subset $Y \subseteq X$, it follows that $\text{sup}(X) \leq \text{sup}(Y)$.

Definition 4. Vertical Database Representation: Given a transaction database D consisting of a set of transactions $\{t_1, t_2, \dots, t_m\}$, where each transaction t_i is a subset of the itemset universe I , the **vertical database representation** transforms D by associating each item $x \in I$ with its corresponding **tidset**, defined as:

$$\mathcal{T}(x) = \{T_i \mid x \in T_i, T_i \in D\}$$

where $\mathcal{T}(x)$ represents the set of transaction identifiers (tids) in which the item x appears.

The vertical representation approach allows efficient intersection-based computation for frequent itemset mining, and there is no distinct candidate generation and support counting phase like in Apriori. Counting is simultaneous with generation (Zaki & Gouda, 2003).

Definition 5. Equivalence classes (Zaki, 2000; Zaki & Gouda, 2003): The equivalence relation partitions the set P into disjoint subsets called equivalence classes. The equivalence class of an element $X \in P$ is given as $[X] = \{Y \in P \mid X \equiv Y\}$. Two itemsets X and Y belong to the same class if they share a common prefix of length k :

$$X \text{ and } Y \in C \iff \text{prefix}_k(X) = \text{prefix}_k(Y)$$

The equivalence classes theory can be utilized to derive candidate itemsets for the next level $l + 1$ from the frequent itemsets of the current level l (which contain itemsets of length l). For instance, suppose we have the following 2-item frequent itemsets at the second level ($l = 2$): $\{1, 3\}, \{1, 4\}, \{1, 5\}, \{2, 3\}, \{2, 4\}, \{4, 5\}$. The itemsets $\{1, 3\}, \{1, 4\}$, and $\{1, 5\}$ share the same prefix $\{1\}$. Candidates for the next level can be generated by joining the prefix with postfixA and postfixB, where postfixA and postfixB are the postfix segments of the level 2 itemsets that do not belong to the shared prefix class. Following this approach $\{1, 3, 4\}, \{1, 3, 5\}, \{1, 4, 5\}$, 3-item itemsets would be generated. Similarly, the itemset $\{2, 3, 4\}$ would be discovered from the equivalence class itemsets with the prefix $\{2\}$. On the other hand, the itemset $\{4, 5\}$ does not belong to any class of equivalence and therefore cannot produce candidate itemsets of length $l + 1$.

4. The proposed HTK-Miner algorithm

4.1. Algorithm overview

HTK-miner is a frequent itemset mining algorithm designed to efficiently discover the top-*k* most frequent itemsets without requiring a predefined minimum support threshold. The algorithm is based on a vertical database representation (VR), where each itemset is associated with a tidset. A tidset is a list of transaction IDs in which it appears. This representation allows support computation through tidset intersections, eliminating the need for complex candidate structures.

HTK-Miner operates in two main phases. The first is called Initialization Phase. In this phase, the algorithm performs a single database scan to construct a vertical representation (VR) and a bidirectional dictionary mapping item names to integer IDs. It then identifies the top-*k* 1-itemsets and initializes a dynamic structure called Q-Heap, which maintains the current top-*k* support values and automatically updates the minimum support threshold during mining. In the second phase, termed the Mining Phase, HTK-Miner employs the vertical data representation (VR) and iteratively generates and evaluates candidate itemsets according to equivalence class theory. The algorithm supports four operational modes, differing in how transactions are represented and intersected. The Q-Heap structure ensures that only itemsets with support exceeding the current minimum are retained, allowing the algorithm to dynamically prune infrequent itemsets. By combining top-*k* mining, vertical representation, equivalent classes and optimized data formats, HTK-Miner aspires to achieve high efficiency and scalability across datasets with varying characteristics.

4.2. Initialization phase

Initially, HTK-Miner utilizes the VR dictionary to efficiently encode the transactional database. Based on Definition 4, the advantage of vertical representation arises because frequent patterns can be counted through tidset intersections, eliminating the need for complex internal data structures. Specifically, candidate generation and support count are achieved in a single step (Zaki, 2000). In VR structure, each itemset serves as a dictionary key, mapped to its corresponding tidset (TS), a list of transaction indexes where the itemset appears (2nd column of Table 5). As will be discussed in Section 4.5, the VR values can be reformatted into an alternative representation format, determined by the operation mode selected by the user among four available options.

HTK-Miner takes as input a transactional database D as shown in Table 2 and a top-*k* integer threshold. In the first step, the algorithm

Table 5
TS and BSN vertical representation of 1-item patterns.

Item (support)	tidset (TS)	bitset	BitSet Number (BSN)
e (6)	[1,2,3,4,5,6]	111111 ₂	63 ₁₀
b (5)	[1,2,4,5,7]	1011011 ₂	91 ₁₀
d (4)	[2,3,5,6]	110110 ₂	54 ₁₀
a (3)	[3,4,7]	1001100 ₂	76 ₁₀
c (2)	[3,5]	10100 ₂	20 ₁₀
f (1)	[1]	1 ₂	1 ₁₀
g (1)	[6]	100000 ₂	32 ₁₀

scans the database once to construct a vertical representation dictionary VR in tidset (TS) format and a bidirectional dictionary, which maps the names of the real items to their corresponding indexes (see Algorithm 1). The bidirectional dictionary is necessary to ensure that the algorithm can uniformly and efficiently handle datasets that do not conform to the FIMI standard (the dataset format used in the workshops on Frequent Itemset Mining Implementations, FIMI'03 (Goethals, 2003) and FIMI'04 (Goethals, 2004), held at IEEE ICDM'03 and IEEE ICDM'04, respectively), which requires the items contained in the dataset to be strictly integer values. From this point onward, the items are represented by their assigned indexes. The bidirectional dictionary will be used later during the output of the frequent itemsets.

Algorithm 1 Vertical dataset construction.

Require: D (dataset file)
Ensure: VR (vertical representation), I (bidirectional item dictionary)

```

1:  $VR \leftarrow \emptyset$   $\triangleright$  Vertical dataset representation
2:  $S \leftarrow \emptyset$   $\triangleright$  Set to ensure unique items
3:  $t \leftarrow 0$   $\triangleright$  Transaction index
4:  $i \leftarrow 0$   $\triangleright$  Item index
5:  $I_{\text{forward}}, I_{\text{reverse}} \leftarrow \emptyset$   $\triangleright$  Bidirectional dictionary
6: for each transaction  $\tau$  in  $D$  do
7:    $t \leftarrow t + 1$ 
8:   for each item  $x \in \tau$  do
9:     if  $x \notin S$  then
10:       $i \leftarrow i + 1$ 
11:       $I_{\text{forward}}[i] \leftarrow x$ 
12:       $I_{\text{reverse}}[x] \leftarrow i$ 
13:       $VR[i, :] \leftarrow [\tau]$ 
14:       $S \leftarrow S \cup \{x\}$ 
15:     else
16:       Append  $t$  to  $VR[I_{\text{reverse}}[x], :]$ 
17:     end if
18:   end for
19: end for
```

4.3. Mining phase

In the next step, the initial top- k dictionary is constructed, storing the top- k 1-item itemsets along with their support values as shown in Algorithm 2. As a result, VR is filtered to keep only the top- k 1-itemsets, discarding any excess items.

This procedure requires the VR created at the previous stage and the user defined top- k threshold. In line 1, the construction of a dictionary ($\mathcal{L}$) takes place, where the key is the item and the value is its support. If the number of 1-item itemsets is smaller than top- k , then $\mathcal{L}$ becomes the initial top- k Frequent 1-Itemset dictionary (lines 2 to 4). Lines 5 to 17 filter the top- k items. The segment between lines 7 and 15 represents a novel mechanism that handles items that are in the k th position of the dictionary, but share the same support value. For example, if the Top-10 itemsets are requested and the itemsets in positions 9, 10, 11, and 12 of the dictionary have exactly the same support value, then 12 itemsets will be returned according to Definition 3.

Algorithm 2 Initial construction of Top- k frequent 1-Itemsets.

Require: $\mathcal{V}_R$ $\triangleright$ Vertical Representation of the dataset
Require: k $\triangleright$ Top- k threshold
Ensure: $\mathcal{T}_K$ $\triangleright$ Dictionary of Top- k 1-itemsets with their support
Ensure: $\mathcal{M}_S$ $\triangleright$ Minimum support threshold

```

1:  $\mathcal{L} \leftarrow \text{Sort} \{ (X, |T_X|) \mid X \in \mathcal{V}_R \}$  in descending order of  $|T_X|$   $\triangleright T_X$  is the TidSet (TS) of  $x$ 
2: if  $|\mathcal{L}| < K$  then
3:   return  $\mathcal{L}, 0$   $\triangleright$  If fewer than  $k$  itemsets exist, return all with  $\mathcal{M}_S = 0$ 
4: end if
5:  $\mathcal{T}_K \leftarrow \emptyset, \mathcal{M}_S \leftarrow -1$ 
6: for Each  $(X, s_X) \in \mathcal{L}$  with index  $i$  do  $\triangleright s_X$  is the support of item  $X$ 
7:   if  $i \geq K - 1$  then
8:     if  $\mathcal{M}_S = -1$  then
9:        $\mathcal{M}_S \leftarrow s_X$ 
10:    else
11:      if  $s_X \neq \mathcal{M}_S$  then
12:        break  $\triangleright$  Stop if support differs
13:      end if
14:    end if
15:  end if
16:   $\mathcal{T}_K[X] \leftarrow s_X$ 
17: end for
18: if  $\mathcal{M}_S = -1$  then  $\triangleright$  The items are less than Top- $k$  threshold
19:    $\mathcal{M}_S \leftarrow 0$ 
20: end if
```

Additionally, a novel structure called Quick Heap (Q-Heap) is initialized to store only the top- k support values, allowing efficient tracking and retrieval of the most frequent itemsets.

4.4. Q-Heap: efficiently increasing min support based on candidate support values

The Q-Heap is a dynamic structure that maintains a top- k list, which functions as a heap to track the top- k support values in real time, always keeping them in descending order. The last value in this heap represents the current minimum support found so far. Whenever a new candidate itemset with support higher than the current minimum is discovered, the Q-Heap updates automatically, recalculating the new minimum support threshold using as few resources as possible and as quickly as possible.

The construction of the Q-Heap is straightforward. It consists of a simple list of integers representing the absolute support values of the top- k itemsets found so far, sorted in descending order. Initialization requires only the support values of the top- k 1-item itemsets found in previous step. Q-Heap features a novel Insert method, as presented in Algorithm 3, which updates its contents while concurrently returning the current minimum support value. The algorithm must be highly efficient as it is executed multiple times throughout the itemset mining process.

Lines 1 to 3 handle initialization. Lines 4 to 9 optimize performance by bypassing the binary search when the new support value matches the current minimum, reducing execution time especially in high k values. Lines 10 to 19 perform a modified binary search insertion, ensuring that the Q-Heap remains sorted in descending order. Finally, lines 20 to 27 manage the heap size, maintaining only the top- k values while returning the updated minimum support threshold.

4.4.1. Complexity analysis - comparison with other strategies

The efficiency of Top- k mining is heavily dependent on the management of the minimum support threshold. We compare three distinct strategies: the proposed Q-Heap from HTK-Miner and HTK-negFIN, the RSC from BTK (Dam et al., 2016), and the Min-Heap (MH) from PTF (Wan & Han, 2024).

Algorithm 3 Insert operation in Q-Heap.

Require: *value* (support value to insert), *heapList* (descending ordered list), *size* (max heap size)

Ensure: Returns the updated minimum support $\mathcal{M}_S$

```

1: low  $\leftarrow$  0
2: high  $\leftarrow$  |heapList| - 1
3: heapLen  $\leftarrow$  high + 1 ▷ Current heap size
4: if heapList[high]  $\geq$  value then
5:   if size = heapLen then
6:     return heapList[high] ▷ Early termination of operation mechanism
7:   else
8:     high  $\leftarrow$  high + 1
9:   end if
10: else
11:   while high < low do
12:     mid  $\leftarrow$   $\lfloor (\textit{high} + \textit{low}) / 2 \rfloor$ 
13:     if heapList[mid] > value then ▷ Descending order comparison
14:       high  $\leftarrow$  mid + 1
15:     else
16:       low  $\leftarrow$  mid
17:     end if
18:   end while
19: end if
20: Insert value at position high in heapList
21: heapLen  $\leftarrow$  heapLen + 1
22: if heapLen > size then
23:   Remove the last element from heapList ▷ Maintain heap size
24:   return heapList[−1] ▷ Return new minimum support
25: else
26:   return 0 ▷ Heap is not full yet, minSup remains 0
27: end if
```

- **Q-Heap:** By maintaining a strictly sorted list of size k , the Q-Heap offers $O(1)$ access to the pruning threshold. Although the worst-case insertion complexity is $O(\log k + k)$ —comprising a binary search to locate the insertion point and an array shift to maintain order—the practical computational cost is significantly reduced by omitting any candidate where $\text{sup}(X) \leq \text{minSup}$. This ensures that the structure is updated only when a genuine increase in the pruning boundary is achievable, thereby maintaining high efficiency in dense datasets or high k values where support ties are frequent.
- **RSC (BTK):** This dictionary-based approach groups itemsets by rank. While it handles “bulk” updates, it suffers from significant memory and computational overhead due to hashing and the management of multiple internal lists, especially in dense datasets where many itemsets share the same support.
- **Min-Heap / MH (PTF):** PTF utilizes a standard binary min-heap. While it provides $O(\log k)$ insertion, the tree-based structure requires constant pointer manipulation or array restructuring (heapify). Furthermore, unlike the Q-Heap, standard heaps often store ties ($\text{sup}(X) = \text{minSup}$), which increases the workload without improving the pruning threshold.

In contrast, our approach decouples tie-handling, allowing the Q-Heap to function as a faster, specialized pruning engine. Table 6 provides a systematic comparison between the proposed Q-Heap, the RSC in BTK and the Min-Heap (MH) in PTF.

4.5. Operation modes

One of the most significant challenges in a data mining algorithm is determining the format in which the data will be represented. With this in mind, HTK-Miner has the capability to represent the transactions of

Table 6

Comparison of minimum support threshold management structures.

Name	Q-Heap	RSC (BTK)	MH (PTF)
Type	top- k	top- k rank	top- k
Data structure	Fixed-size Sorted List	Dictionary of Lists	Binary Min-Heap
Threshold	$O(1)$	$O(1)$	$O(1)$
access	(Tail element)	(Min Key)	(Root element)
Insertion strategy	$O(\log k + k)$	$O(1)$ hash + $O(m)$ append	$O(\log k)$
Tie-Handling	Omitted	Stored	Stored
Pruning dynamics	Continuous & Aggressive	Group-based (Rank)	Continuous
Memory efficiency	Highest	Moderate	Moderate
Runtime	(Linear)	(Hashing)	(Pointers)
gain	High	Lower	Lower
		bucket overhead	$O(\log k)$ overhead

Table 7vertical representation of 2-item itemsets for $K = 10$.

Itemset (support)	TidSet TS	BitSet bitset	Number BSN	diffset DTS	Difference bitset	DBSN
e,d (4)	[2,3,5,6]	110110 ₂	54 ₁₀	[1,4]	1001 ₂	9 ₁₀
e,b (4)	[1,2,4,5]	11011 ₂	27 ₁₀	[3,6]	100100 ₂	36 ₁₀
d,c (2)	[3,5]	10100 ₂	20 ₁₀	[2,6]	100010 ₂	34 ₁₀
b,a (2)	[4,7]	1001000 ₂	72 ₁₀	[1,2,5]	10011 ₂	19 ₁₀
b,d (2)	[2,5]	10010 ₂	18 ₁₀	[1,4,7]	1001001 ₂	73 ₁₀
e,a (2)	[3,4]	1100 ₂	12 ₁₀	[1,2,5,6]	110011 ₂	51 ₁₀
e,c (2)	[3,5]	10100 ₂	20 ₁₀	[1,2,4,6]	101011 ₂	43 ₁₀

Table 8vertical representation of 3-item itemsets for $K = 10$.

Itemset (support)	TidSet TS	BitSet bitset	Number BSN	diffset DTS	Difference bitset	DBSN
e,d,b (2)	[2,5]	10010 ₂	18 ₁₀	[3,6]	1001 ₂	9 ₁₀
e,d,c (2)	[3,5]	10100 ₂	20 ₁₀	[2,6]	100010 ₂	34 ₁₀

an itemset in four distinct formats. These include: as a list of transaction IDs (tidsets), for example [2, 3, 8, 9, 11]; as an integer where each bit position indicates the presence or absence of the itemset in the corresponding transaction ID (e.g., 1414₁₀. See Section 4.5.2 for details); as a list of differences between transaction IDs (diff-lists); and as an integer representing the differences between transaction IDs per itemset. To handle this, the algorithm operates in four distinct modes:

- **Vertical representation (VR) with TidSets (TS)**
- **Vertical representation (VR) with BitSet Numbers (BSN)**
- **Vertical representation (VR) using the Differences in TidSets (DTS)**
- **Vertical representation (VR) using the Differences of the bitset numbers (DBSN)**

4.5.1. Vertical representation with TidSets (TS)

The vertical format maintains, for each item, its transaction ID set (tidset), which consists of all transaction IDs (tids) where the item appears (Zaki, 2000; Zaki & Gouda, 2003). Consequently, considering the transactional data from Table 2 as a reference, its VR structure is shown in columns 1 and 2 of the Table 5. The (TS)-based Vertical Representation is utilized in the initial stage of the algorithm in all modes, as this format enables the rapid conversion of horizontal representation into vertical representation. When it is necessary to identify the transactions and the absolute support of an itemset at the next level, an intersection is performed between the tidsets involved in the previous level using the equivalence classes theory, as mentioned in Definition 5. In our running example and according to Table 5, to find the transactions and absolute

support of (b, d):

$$TS(b, d) = TS(b) \cap TS(d) = [1, 2, 4, 5, 7] \cap [2, 3, 5, 6] = [2, 5]$$

$$Supp(b, d) = length([2, 5]) = |5 - 2| = 2$$

4.5.2. Vertical representation with bitset numbers (BSN)

As mentioned above, the initial VR construction is in tidset (TS) format. The reason why the initial vertical representation of the data is done in the form of TS and not BSN, even when the bitset mode has been selected, is that maintaining BSN is computationally expensive task. This ultimately causes significant delays during the reading of the transactional dataset. The conversion from TS to BSN is performed at a later stage, only for the top- k 1-item itemsets and only if the bitset mode has been activated. The BSN reduces memory use by one to two magnitudes, while significantly accelerating intersections and other bitwise operations needed between bitset numbers. Furthermore, the BSN transformation is facilitated through a multi-process parallelization strategy (as formalized in Algorithm 4), which is engineered to minimize computational overhead and accelerate execution, as detailed in Section 4.6.

According to Table 5, the item a appears in transactions 3, 4, and 7 (2nd column in Table 5). To represent this as a BitSet Number (BSN), we create a binary number where the 3rd, 4th, and 7th bits on the right are set to 1, while all other bits remain 0. This results in the binary value 1001100_2 (3rd column in Table 5), which corresponds to the decimal number 76_{10} and represents the BSN equivalent of the tidset (4th column in Table 5). Likewise in TS, when it is necessary to identify the transactions and the absolute support of an itemset at the next level, a bitwise AND is performed between the bitset number involved in the previous level. In our running example, to find the transactions and absolute support of (b, d):

$$BSN(b, d) = BSN(b) \wedge BSN(d) = 91_{10} \wedge 54_{10}$$

$$= 1011011_2 \wedge 110110_2 = 10010_2 = 18_{10}$$

$$Supp(b, d) = bitCount(10010_2) = 2$$

where *bitCount* is the count of the BSN bits that are set.

4.5.3. Vertical representation (VR) using the differences in tidsets (DTS)

Another mode supported by the HTK-Miner is storing the differences of transaction ids named DiffsetTS (DTS). The concept of diffsets, originally introduced in Zaki and Gouda (2003) and utilized in the dEclat and dCharm algorithms, has emerged as one of the most effective vertical data representation techniques. It has been widely adopted in numerous algorithms for mining frequent itemsets and related tasks (Zaki & Hsiao, 2002). The logic behind diffsets lies in storing the differences between the tidsets in memory rather than the tidsets themselves. Diffsets can dramatically reduce the memory consumption in dense data. However, the exact opposite occurs in sparse data. According to Zaki and Gouda (2003), it is feasible to perform mining at a certain level using tidsets and then convert the representation to diffset format in subsequent levels. Based on this observation, the DTS mode of the HTK-Miner algorithm was designed to use tidsets at the first level and switch to the diffset representation from the second level onward. This approach was adopted because directly converting the original transactional data into diffset format is computationally intensive. Moreover, using diffsets at the initial stage would theoretically be more efficient only for datasets with a density significantly greater than 50% (taking into account the additional computational time required to directly represent 1-itemsets as diffsets). Such datasets are rarely encountered in real-world scenarios. Finding the transactions and absolute support of the next level itemset depends on the level at which the mining is being performed. At level 1, the data representation uses tidsets. To find the diffset of a 2nd-level itemset, the following operations must be performed:

According to the definition given by Zaki and Gouda (2003), if there is e class with prefix P then for all members of the class PX and PY it holds that $DT S(PXY) = TS(PX) - TS(PY)$, where $DT S(PXY)$ is the diffset of the itemset P, X, Y and $TS(PX), TS(PY)$ are the tidsets of the

itemsets P, X and P, Y , respectively. Similarly, for the absolute support, the relationship $Supp(PXY) = Supp(PX) - |DT S(PXY)|$ holds. Return to the running example (Tables 5,7):

$$DT S(b, d) = TS(b) - TS(d) = [1, 2, 4, 5, 7] - [2, 3, 5, 6] = [1, 4, 7]$$

$$Supp(b, d) = Supp(b) - |DT S(b, d)| = 5 - 3 = 2$$

In subsequent levels, only diffsets are utilized. According to the definition in Zaki and Gouda (2003), $DT S(PXY) = DT S(PY) - DT S(PX)$ holds. In the running example (Tables 5 and 7):

$$DT S(e, d, c) = DT S(e, c) - DT S(e, d) = [1, 2, 4, 6] - [1, 4] = [2, 6]$$

$$Supp(e, d, c) = Supp(e, d) - |DT S(e, d, c)| = 4 - 2 = 2$$

4.5.4. Vertical representation (VR) using the differences of the bitset numbers (DBSN)

The last mode of HTK-Miner is also based on the idea of diffsets (refer to Section 4.5.3). However, the representation is not done using lists as in the DTS mode, but with numbers called Diffset BitSet Numbers (DBSN). Similar to the BSN mode, the conversion from TS to BSN is carried out at a later stage, only for the top- k 1-item itemsets due to high computational cost. The DBSN mode significantly reduces memory usage by one to two orders of magnitude while greatly accelerating the essential bitwise operations between bitset numbers. Moreover, the BSN transformation employs a multi-process strategy (Algorithm 4) to drastically reduce execution latency, a methodological advancement further elaborated in Section 4.6. Subsequently, the conversion from BSN to DBSN takes place during the calculation of the candidate 2-item itemsets. In the subsequent levels, bitwise operations are performed between DBSNs.

Calculating the difference between two DBSNs requires two bitwise operations: a bitwise XOR operation followed by a bitwise AND operation. To determine the DBSN of a 2nd-level itemset, the following steps must be executed (Tables 5,7):

$$DBSN(b, d) = BSN(b) \wedge (BSN(b) \oplus BSN(d))$$

$$= 1011011_2 \wedge (1011011_2 \oplus 110110_2) = 1001001_2 = 73_{10}$$

$$Supp(b, d) = Supp(b) - bitCount(1001001_2) = 5 - 3 = 2$$

where *bitCount* is the count of DBSN bits that are set.

On the other hand, to calculate the transactions and support of itemsets at subsequent levels, the following operations must be performed (Tables 7 and 8):

$$DBSN(e, d, c) = DBSN(e, c) \wedge (DBSN(e, c) \oplus DBSN(e, d))$$

$$= 101011_2 \wedge (101011_2 \oplus 10010_2) = 100010_2 = 34_{10}$$

$$Supp(e, d, c) = Supp(e, d) - bitCount(100010_2) = 4 - 2 = 2$$

4.6. Contributions and implementation specifics

The performance of FIM algorithms is primarily determined by three critical pillars: the search paradigm employed for pattern discovery, the efficiency of the underlying data representation, and the specific optimization techniques utilized during implementation (Bodon, 2003). We show that data structure selection as well as implementation tuning are at least as important as the first factor mentioned. Here is a reference for the data structures and implementation tuning techniques used to enhance the effectiveness of the HTK-Miner algorithm.

To begin, the initial **transactional data file is read only once** to directly create a vertical database representation (VR) format. The transactional dataset is not read again thereafter.

A **bidirectional dictionary** is employed to map items to integer indices, thereby improving the efficiency of the algorithm. The original item identifiers are restored only during the extraction of the top- k itemsets. This strategy accelerates execution, reduces memory requirements, and enhances scalability, enabling the algorithm to efficiently handle raw transactional data.

The vertical database is stored in a simple dictionary of lists, avoiding the use of the popular pandas dataframe structure, which does not handle transactional data (1-MBL type datasets (Malliaridis et al., 2024)) efficiently, or other complex and potentially memory intensive data structures.

As mentioned before, HTK-Miner is based on the theory of **equivalence classes** (Zaki, 2000; Zaki & Gouda, 2003), where for each frequent itemset at level l , the tids and support of itemsets at the next level $l + 1$ can be computed. These sets of items share all the items except the last one, which also belongs to the frequent itemsets of level l . This approach allows the vertical representation (VR) of levels preceding level l to be discarded from memory. **Storing only the last level l** , which is required to identify the itemsets of the next level $l + 1$, led to a reduction in required memory by approximately **two orders of magnitude**.

For HTK-Miner core calculations, **an iterative approach was preferred over recursion**. Recursion requires storing the data of the current level in the call stack with each procedure call, which places a significant burden on system resources. However, using encapsulated multiple iterations provides more explicit control over loop variables and state, further optimizing the procedure. Additionally, with recursion, it would be impossible to store only the last level of VR, as explained in the previous paragraph.

To raise the current *minSup* as quickly as possible, a **Smart Iteration MEchanism (SIME)** is employed to combine the equivalent classes in such a way that **combinations with the highest support are examined earlier**. With this approach, the algorithm performance improved by 5% to 60%, while also achieving memory savings of 2% to 70%, depending on the mode of the algorithm and the input data (see Section 4.7 for more details).

After conducting research, we concluded that **tuples are the better choice for dictionary keys compared to strings**, as they made the algorithm's operation more efficient and scalable. In simple terms, consider the set of items 2, 5, 7, where the numbers represent the indices of the items. Representing the keys of internal structures as tuples (e.g., (2, 5, 7)) is more efficient than using string formats such as "2,5,7".

The use of **BitSet Numbers (BSN)** – both as a replacement for tidset lists and diffset lists in vertical data representation – had a catalytic impact on the performance of HTK-Miner. Based on experimental results (refer Section 7), the retrieval speed of frequent itemsets using BSN, in certain cases and particularly for large top- k thresholds (e.g. 10,000), exceeded improvements by **two orders of magnitude**.

Moreover, the initial transformation of BSNs from TSs (used in BSN and DBSN modes), although a computationally expensive process, can be efficiently implemented using **parallel programming techniques**. The proposed transformation is inherently parallelizable, as the conversion of each tidset into its corresponding BSN is a decoupled operation that eliminates the need for shared resources or synchronization primitives. Compared to the sequential approach, our multi-threaded parallelization achieved speedup factors ranging from 1× to 6× on a 6-core/12-thread configuration for this task. This acceleration is directly proportional to the total computational workload ($|T| \times Avg_Size$), demonstrating that **the parallel efficiency scales effectively with the volume of the transactional data**. Furthermore, the task-independent nature of this conversion suggests high scalability, indicating that performance could be further enhanced by leveraging massively parallel architectures, such as General-Purpose Graphics Processing Units (GPGPUs) or high-density multi-core CPU clusters. The parallel programming model employed is formalized in Algorithm 4. This approach leverages the inherent task independence of the conversion process, allowing for a **lock-free parallel execution**.

The HTK-Miner returns the top- k frequent itemsets with one exception: It also **includes any additional frequent itemsets that share the exact same support as the k th itemset**, following the mechanism described in lines 5 to 17 of Algorithm 2. This approach ensures that no useful information is excluded from the mining process while simultaneously enabling a direct comparison of its results with those of algorithms

Algorithm 4 Parallel BSN or DBSN transformation.

Require: Vertical Dictionary $vR = \{(key, TS_1), (key, TS_2), \dots\}$, where TS is a list of tids (tidset)

Ensure: Vertical Dictionary $vBS = \{(key, BS_1), (key, BS_2), \dots\}$, where BS is a BSN or DBSN number.

```

1: for each  $(key, TS) \in vR$  do in parallel do
2:    $vBS[key] \leftarrow \bigvee_{i \in L} (2^i)$            ▷ Aggregate bits via bitwise OR
3: end for
4: return  $vBS$ 

```

that operate using a minimum support threshold or top- k Rank. In addition to other statistics, the algorithm extracts the relative support and the top- k rank identified. These statistics can be directly provided as input to the aforementioned algorithms, which will produce exactly the same frequent itemsets.

The use of the **Q-Heap structure**, as explained in Section 4.4, led to a further reduction in search space. Its implementation allowed the rapid elevation of the minimum support threshold in real time during the mining process. This novel lightweight structure was also utilized in the HTK-negFIN algorithm (see Section 5), and it can easily be integrated into any top- k itemset mining algorithm.

Although the Q-Heap operates in real-time, the synchronization of the TopKFI dictionary occurs at the end of each level in the breath first approach to conserve system resources. At this point, the itemsets of the current level are detected and merged with those of previous levels. **The remaining itemsets at the current level are obsolete and, consequently, discarded from Vertical Representation (VR)**. According to our calculations, this approach improved the algorithm's performance by at least 5 percent.

Ultimately, various minor optimizations to the algorithm were made as a final refinement. The iteration of the set of items at the current level, where the equivalence class theory is applied, is performed using a tuple structure, instead of a list, because as an immutable structure, it is faster in iterations while also consuming less memory (line 7 in Algorithm 5). Additionally, the number of itemsets at the current level is stored in a variable rather than being recalculated in each subsequent iterative process. This modification significantly accelerated the algorithm, since accessing the count of itemsets can exceed thousands or, in the case of very large datasets, even reach one million (line 8 in Algorithm 5). In summary, during each iteration, a check is performed to determine whether either of the two itemsets - to which the equivalence class theory is applied - has exceeded the dynamically changing minimum support threshold through the operation of the Q-Heap (lines 10–11, 14–15 in Algorithm 5). These improvements, when combined, led to a reduction in the execution time of HTK-Miner by approximately fifty percent.

4.7. The HTK-Miner algorithm

In Algorithm 5 the pseudocode of HTK-Miner is presented. In line 1, the vertical data representation dictionary VR is constructed in tidset format (see Algorithm 1). Next, the initial 1-item top- k frequent itemsets are discovered, along with the current minimum support threshold (*minSup*) and the Q-Heap instantiation in Line 2 (see Algorithm 2). If a bitset mode has been selected, the VR is transformed into the BSN format. The parallel processing mechanism for converting the vertical representation from TS format to BSN (line 3 of the algorithm) is presented in Algorithm 4.

The algorithm follows a breadth-first approach, starting from level 1 and progressively moving to subsequent levels (lines 5–34). To traverse the itemsets of each level l (where $cLTOPK$ contains only the itemsets of the current level l along with their corresponding supports, sorted in descending order by support), two pointers, iA and iB , are used to generate candidate itemsets for the next level $l + 1$ based on the equivalence class theory. The first pointer, iA , is initially set to the most

Algorithm 5 HTK-Miner algorithm.

Require: D $\triangleright$ A transactional database file
Require: k $\triangleright$ Top- k itemsets threshold defined by user
Require: $Mode$ $\triangleright$ 1) TS, 2) BSN (default), 3) DTS, 4) DBSN
Ensure: $topKFI$ $\triangleright$ Final Top- k frequent itemsets with their support value

```

1:  $VR, I \leftarrow VerticalDatasetConstruction(D)$   $\triangleright$  Algorithm 1
2:  $TopKFI, minSup, QHeap \leftarrow InitialTopKFI(VR, K)$   $\triangleright$  Algorithm 2
3: if  $mode == 2$  or  $mode == 4$  then  $VR = BSN(VR)$   $\triangleright$  TS to BSN
4:  $cLTopK \leftarrow TopKFI, level \leftarrow 1$ 
5: while  $level > 0$  do
6:    $nextlevelData \leftarrow \emptyset, nextLevelTopK \leftarrow \emptyset$ 
7:    $tupleOfKeys \leftarrow cLTopK.keys$ 
8:    $countKeys \leftarrow |tupleOfKeys|$ 
9:   for  $iB \leftarrow 1$  to  $countKeys - 1$  do
10:     $classB \leftarrow tupleOfKeys[iB]$ 
11:    if  $cLTopK[classB] < minSup$  then
12:      break
13:    for  $iA \leftarrow 0$  to  $iB - 1$  do
14:       $classA \leftarrow tupleOfKeys[iA]$ 
15:      if  $cLTopK[classA] < minSup$  then
16:        break
17:       $prefixA \leftarrow classA[:-1], prefixB \leftarrow classB[:-1]$ 
18:      if  $prefixA = prefixB$  then  $\triangleright$  Algorithm 5
19:         $tr, supp \leftarrow NextCandidate(VR, classA, classB, mode)$ 
20:        if  $supp \geq minSup$  then
21:           $minSup \leftarrow QHeap.insert(supp)$   $\triangleright$  Algorithm 3
22:           $key \leftarrow prefixA + (classA[-1], classB[-1])$ 
23:           $nextlevelData[key] \leftarrow transactions$ 
24:           $nextLevelTopK[key] \leftarrow supp$ 
25:        end if
26:      end if
27:    end for
28:  end for
29:   $Filter: nextLevelTopK \leftarrow \{(k, v) | v \geq minSup\}$ 
30:   $Merge: topKFI \leftarrow topKFI \cup nextLevelTopK$   $\triangleright$  Algorithm 7
31:   $topKFI, minSup, cLTopK \leftarrow getTopKFI(topKFI, level + 1)$ 
32:  if  $cLTopK = \emptyset$  then  $level \leftarrow 0$ 
33:  else  $level \leftarrow level + 1, VR \leftarrow nextlevelData$ 
34: end while
35: return  $topKFI, minSup$ 

```

frequent itemset of the level, while the second pointer, iB , is set to the next one. To raise the current $minSup$ as quickly as possible, the equivalent classes are combined in such a way that itemset combinations with the highest $minSup$ are prioritized (lines 9–13) using the **Smart Iteration MEchanism (SIME)**, illustrated in the following example. Consider the 1-itemsets 1, 2, 3, 4, and 5 ordered by their support count value in descending order. According to the SIME approach, the order of candidate 2-item itemsets would be as follows:

{1, 2}, {1, 3}, {2, 3}, {1, 4}, {2, 4}, {3, 4}, {1, 5}, {2, 5}, {3, 5}, {4, 5}
 instead of:

{1, 2}, {1, 3}, {1, 4}, {1, 5}, {2, 3}, {2, 4}, {2, 5}, {3, 4}, {3, 5}, {4, 5}

In simple words, itemset combinations with the highest support values are prioritized as candidate itemsets of the next level. This approach resulted in performance improvements ranging from 5% to 60%, and memory savings between 2% and 70%, depending on the algorithm mode and the input data, as demonstrated in our experiments presented in Section 7.

After verifying that none of the participating itemsets has a support lower than the current $minSup$ (lines 11, 12 and 15, 16), a check is performed to determine whether they belong to the same equivalence class (lines 17–18). For example, the itemsets {2, 4, 6} and {2, 4, 8} belong to

Algorithm 6 Next candidate.

Require: $CLVR$ $\triangleright$ Current level Vertical Representation
Require: $ClassA, ClassB$ $\triangleright$ Equivalent classes to be processed
Require: $Mode$ $\triangleright$ 1) TS, 2) BSN (default), 3) DTS, 4) DBSN
Require: $level$ $\triangleright$ $level$ and $cLTopK$ are required in modes 2, 4
Require: $cLTopK$
Ensure: $tr, support$ $\triangleright$ tr are the itemset transactions

```

1: if  $mode = 1$  then  $\triangleright$  Intersection mode with TS
2:    $tr \leftarrow CLVR[classA] \cap CLVR[classB]$ 
3:    $support \leftarrow \text{length of } transactions$ 
4: else if  $mode = 2$  then  $\triangleright$  Intersection mode with BSN
5:    $tr \leftarrow CLVR[classA] \wedge CLVR[classB]$ 
6:    $support \leftarrow \text{bit\_count}(transactions)$   $\triangleright$  Count of set bits in BSN
7: else if  $mode = 3$  then  $\triangleright$  diffset mode with TS
8:   if  $level = 1$  then  $\triangleright$  Level 1 works with TS
9:      $tr \leftarrow CLVR[classA] - CLVR[classB]$ 
10:  else  $\triangleright$  In next levels, TS is transformed to DTS
11:     $tr \leftarrow CLVR[classB] - CLVR[classA]$ 
12:     $support \leftarrow cLTopK[classA] - \text{len}(transactions)$ 
13:  else  $\triangleright$  diffset mode with BSN
14:    if  $level = 1$  then  $\triangleright$  Level 1 works with BSN
15:       $tr \leftarrow CLVR[classA] \wedge (CLVR[classA] \oplus CLVR[classB])$ 
16:    else  $\triangleright$  In next levels, BSN is transformed to DBSN
17:       $tr \leftarrow CLVR[classB] \wedge (CLVR[classB] \oplus CLVR[classA])$ 
18:     $support \leftarrow cLTopK[classA] - \text{bit\_count}(transactions)$ 
19:  end if
20: return  $tr, support$ 

```

the same equivalence class because they share the same prefix {2, 4}, whereas {3, 4, 6} and {3, 5, 6} do not.

In line 19, the transactions and support of the candidate itemset for the next level are calculated based on the process described in Algorithm 6. The operation of Algorithm 6 is straightforward and depends on the mode selected by the user. If a candidate's support value is found to be greater than or equal to $minSup$, it is considered frequent and is inserted into both the $nextLevelData$ dictionary (which stores the transactions of the next level itemsets) and the $nextLevelTopK$ dictionary (which stores the support values of the next level itemsets).

After processing all potential itemsets of the current level l , the top k itemsets discovered at the next level $l + 1$ are filtered by discarding the rest in line 29, and are merged with the frequent itemsets from previous levels (line 30). Immediately afterward, the $getTopKFI$ process, described in Algorithm 7, is executed on line 31. This process returns the current dictionary of top- k itemsets and the updated $minSup$, as well as the frequent itemsets of level $l + 1$ that were recently recovered for use in discovering itemsets of the next level, $l + 2$ (lines 32–33).

The Algorithm 7 is nearly identical to the Algorithm 2, with only minor modifications. Firstly, it does not involve updating the Q-Heap structure at all, as it is already synchronized in real time. In addition, it returns a dictionary containing the Top- N itemsets that belong to the current level l ($cLTopK$), which are then used to discover the corresponding itemsets of the next level $l + 1$. It is evident that $K > N$, since K represents the number of frequent itemsets at all levels from 1 to l , while N represents the number of frequent itemsets only at current level l .

As shown in line 35, Algorithm 5 returns the top- k frequent itemsets along with their respective support counts and the final minimum support threshold reached during the execution.

5. The HTK-negFIN proposed algorithm

In this section, we present HTK-negFIN, an extension of the negFIN algorithm (Aryabarzan et al., 2018) for frequent itemset mining. Although negFIN operates using a minimum support threshold,

Algorithm 7 getTopKFI.**Require:** $topKItemSets, level, K$ **Ensure:** $topKDict, minSup, cLTopK$

```

1:  $SDict \leftarrow Sort(topKItemSets)$   $\triangleright$  Sorted in descending order of support
2:  $topKDict \leftarrow \emptyset, cLTopK \leftarrow \emptyset, mS \leftarrow -1$ 
3: for each  $(index, key, value)$  in  $SDict$  do
4:   if  $index \geq K - 1$  then
5:     if  $mS == -1$  then  $mS \leftarrow value$ 
6:     else if  $value \neq mS$  then break
7:   end if
8:    $topKDict[key] \leftarrow value$ 
9:   if  $len(key) == level$  then
10:     $cLTopK[key] \leftarrow value$ 
11:   end if
12: end for
13: if  $mS == -1$  then  $minSup \leftarrow 0$ 
14: Else  $minSup \leftarrow mS$ 
15: return  $topKDict, minSup, cLTopK$ 

```

HTK-negFIN introduces a crucial modification: it accepts a k parameter value to discover the top- k frequent itemsets, aligning with the methodology of the HTK-Miner algorithm presented in Section 4.

5.1. The negFIN algorithm

NegFIN follows the pattern-growth approach and is efficient in both runtime and memory consumption when given a minimum support threshold. The algorithm initially scans the input data to create a table L_1 containing the frequent itemsets of 1 item, sorted in ascending order based on their support. A second scan of the database follows to construct a tree-like structure called BMC-tree (bitmap code tree), where each itemset P_k is represented by a bitmap code of size nf (the total number of frequent 1-item itemsets), as follows:

$$BMC(P_k) = b_{nf-1} \dots b_1, b_0$$

The j th item in L_1 is assigned to bit b_j in the bitmap. If an item $i \in L_1$ is a member of P_k , its corresponding bit is set to 1; otherwise, it is set to 0. The root of BMC-tree has no items but has prefix subtrees as children. Each node of a subtree holds an item $i \in L_1$. If the father of this node represents an item j , then $j > i$ and the segment of the path leading to this node is represented by the itemset $node_path$. Each node of BMC-tree consists of four fields:

- **item_name** which holds an item $i \in L_1$
- **count** which is the support of the itemset
- **bitmap_code** which holds $BMC(node_path)$
- **children_list** which holds all the children of this node

If a node N holds an item i_1 , and a node $N.father$ be its father node, then $N.bitmap_code = N.father.bitmap_code \vee 2^{index(i_1)}$.

The pair of fields $bitmap_code$ and $Count$ of a BMC-Tree node, called N_Info is used to form $Nodesets$. The $Nodeset$ of an itemset P_k is the set of all N_info pairs of a BMC-Tree nodes that hold an item $i_1 \in P_k$ and each item in $i_k, i_k - 1 \dots i_2$ is held in one of the ancestor nodes (Aryabarzan et al., 2018). According to this:

$$Nodeset(P_k) = \left\{ \begin{array}{l} N \text{ holds } i_1, \text{ and } \forall i_j, 1 \leq j \leq k, \\ \text{The } N_info \text{ of } N \mid \text{the bit assigned to } i_j \text{ in} \\ \quad \quad \quad N.bitmap_code \text{ is } 1 \end{array} \right\}$$

The NegNodeset of the itemset P_k is a set of all the N_info pairs of nodes in the BMC tree that hold the item $i_1 \in P_k$, each item in $i_{k-1} \dots i_2$ is held in one of their ancestor nodes, and the item i_k is not held in any

of their ancestor nodes. According to this:

$$\begin{aligned} NegNodeset(P_k = i_k i_{k-1} \dots i_2 i_1) &= Nodeset(P'_k = \neg i_k i_{k-1} \dots i_2 i_1) \\ &= \left\{ \begin{array}{l} N \text{ holds } i, \forall i_j, 1 \leq j \leq k-1, \\ \text{The } N_info \text{ of } N \mid \text{the bit assigned } i_j \text{ in } N.bitmap_code \text{ is } 1. \\ \quad \quad \quad \text{and the bit assigned } i_k \text{ is } 0 \end{array} \right\} \end{aligned}$$

For example, if $Nodeset(bd)$ is represented by two N_info pairs like:

$$Nodeset_{bd} = \{(01010, 1), (11110, 1)\}$$

then,

$$Nodeset_{\neg abd} = NegNodeset_{abd} = \{(01010, 1)\}$$

and

$$Nodeset_{abd} = \{(11110, 1)\}$$

Furthermore, in Aryabarzan et al. (2018) it is shown that:

$$support(abd) = support(bd) - support(\neg abd) = 2 - 1 = 1$$

To continue, negFIN employs a set-enumeration tree called Frequent Itemset Tree (FI-Tree) to represent the search space. The algorithm works in three main phases. First, it builds the BMC-Tree, finds all frequent single items (1-itemsets) with their Nodesets, and creates the first level of the FI-Tree. Second, it identifies frequent item pairs (2-itemsets) with their NegNodeSets while constructing the FI-Tree's second level. Finally, it finds all larger item groups (k -itemsets where $k \geq 3$) with their NegNodeSets to complete the remaining FI-Tree levels.

NegFIN algorithm uses the superset equivalence property introduced in Deng and Lv (2014), to efficiently prune unnecessary itemsets during this process. According to this, given two itemsets P and Q and an item i , where $P \cap Q = \emptyset, i \notin P$ and $i \notin Q$, if $support(P_k) = support(P \cup \{i\})$, then $support(P \cup Q) = support(P \cup Q \cup \{i\})$.

The negFIN algorithm served as the basis for developing the HTK-negFIN algorithm, which takes the parameter k as input to identify the frequent itemsets of k in the top instead of requiring a minimum support threshold.

5.2. The HTK-negFIN algorithm

Algorithm 8 The HTK-negFIN algorithm.**Require:** D (transactional database), k (Top- k threshold)**Ensure:** F (Top K frequent itemsets)

```

1:  $F = \emptyset$ 
2:  $BMC, L_1, minSup, QHeap \leftarrow construct\_BMC\_tree(D, K);$   $\triangleright$  Algorithm 9
3:  $F = F \cup L_1$ 
4: for each node  $N$  in the BMC-tree do
5:   Append the  $N\_info$  of  $N$  into the  $Nodeset$  of item  $N.item\_name$ 
6: end for
7:  $root.level = 0;$   $\triangleright$  Create the root node at level 0
8:  $root.children\_list = \emptyset$ 
9:  $root.item\_name = \emptyset$ 
10:  $root.itemset = \emptyset$ 
11: for each item  $i \in L_1$  do
12:   Create the node  $child_i$ 
13:    $child_i.level = root.level + 1$ 
14:    $child_i.item\_name = i$ 
15:    $child_i.itemset = \{i\}$ 
16:   Append  $child_i$  into  $root.children\_list$ 
17:    $F = F \cup Construct\_FI\_Tree(child_i, \emptyset, minSup)$   $\triangleright$  Algorithm 11
18: end for
19:  $F \leftarrow getTopKFI2(F, K)$   $\triangleright$  Algorithm 10

```

The key difference between HTK-negFIN and negFIN lies in their input parameters. HTK-negFIN requires the parameter k to identify the

top- k frequent itemsets, while negFIN relies on the minimum support threshold. Unlike HTK-Miner, which generates candidate frequent itemsets, both HTK-negFIN and negFIN belong to the pattern-growth family of algorithms. They employ tree-based structures to compactly represent input data, significantly reducing memory usage while simultaneously improving computational efficiency. The development of HTK-negFIN was inspired by Abdelaal et al. (2021), which modified the PREPOST, PREPOST+, and FIN algorithms to operate with the k parameter to extract frequent itemsets of the top- k rank. This section presents the adaptations made to negFIN with explicit highlighting of all modified algorithm components and functions that differ from the original implementation. Note that this work will not elaborate on the similarities between the two algorithms.

HTK-negFIN employs the Q-Heap structure (see Section 4.4) to efficiently raise the minimum support threshold, which is initially set to 0 - identical to the HTK-Miner approach. It also incorporates a function similar to *getTopKFI* described in Algorithm 7, called *getTopKFI2*, to retrieve: (a) the frequent top- k itemsets and (b) any additional itemsets that share identical support with the top- k itemset. HTK-negFIN (see Algorithm 8) requires as input the initial data in 1-MBL format (transactional data) (Malliaridis et al., 2024) and the parameter k . In the initial stage (line 2), it invokes a modified *construct_BMC_tree* function to build the BMC-Tree.

5.2.1. The modified BMC tree construction

Algorithm 9 HTK-negFIN procedure *construct_BMC_Tree*.

Require: D (transactional database), k (Top- k threshold)
Ensure: BMC (BMC-tree), L_1 (ordered item list by their support)
Ensure: $minSup$ (Current minimum support), $QHeap$

```

1: Scan  $D$  to find  $F_1$ ;
2:  $L_1, minSup, QHeap = getTopKFI2(F_1, K)$   $\triangleright$  Algorithm 10
3:  $BMC.item-name = \emptyset$ ;  $BMC.count = 0$ ;  $\triangleright BMC = \text{root of the BMC-tree}$ 
4:  $BMC.bitmap-code = b_{nf-1}, \dots, b_1, b_0$ , where  $b_i = 0$ , and  $0 \leq i \leq nf - 1$ 
5: for each transaction  $T$  in  $D$  do
6:   Remove all infrequent items from  $T$  based on  $minSup$ 
7:   Sort  $T$  items according to the order of items in reverse order of  $L_1$ 
8:    $current-root = BMC$ ;
9:   for each item  $i$  in  $T$  do  $\triangleright$  Insert  $T$  into BMC-tree
10:    Let  $N$  be a child of  $current-root$ , in such a way that  $N.item-name = i$ 
11:    if such node does not exist then
12:      Create the new node  $N$ 
13:       $N.item-name = i$ 
14:       $N.count = 0$ 
15:      Add  $N$  to  $current-root.children-list$ 
16:    end if
17:     $N.count = N.count + 1$ 
18:     $N.bitmap-code = current-root.bitmap-code \vee 2^{index(i)}$ 
19:     $current-root = N$ 
20:  end for
21: end for
22: Return  $BMC, L_1, minSup, QHeap$ 

```

In the BMC construction function (Algorithm 9), the transactional data and the top- k threshold are required to ensure a BMC-Tree, an ordered list of items according to their support in descending order, an initialized temporary minimum support value $minSup$ and the Q-Heap instantiation.

The algorithm requires exactly two passes over the input data. The first pass computes the support counts for all elements and is followed by the *getTopKFI2* function to ensure that the top k elements are ordered

Algorithm 10 HTK-negFIN procedure *getTopKFI2*.

Require: $topKItemSets, K$
Ensure: $topKDict, minSup, QHeap$

```

1:  $SDict \leftarrow Sort(topKDict)$   $\triangleright$  Sorted in descending order of support
2:  $topKDict \leftarrow \emptyset, cLTopK \leftarrow \emptyset, mS \leftarrow -1$ 
3: for each ( $index, key, value$ ) in  $SDict$  do
4:   if  $index \geq K - 1$  then
5:     if  $mS == -1$  then  $mS \leftarrow value$ 
6:     else if  $value \neq mS$  then break
7:   end if
8:    $topKDict[key] \leftarrow value$ 
9: end for
10: if  $mS == -1$  then  $minSup \leftarrow 0$ 
11: Else  $minSup \leftarrow mS$ 
12:  $QHeap \leftarrow topKDict.values$   $\triangleright$  filled with topK support values
13: return  $topKDict, minSup, QHeap$ 

```

in descending order by their support, the first minimum support value $minSup$ threshold and the instantiation of a Q-Heap structure (lines 1–2 in Algorithm 9).

The *getTopKFI2* function, presented in Algorithm 10, is a specialized component of the *getTopKFI* in Section 4.7, preserves the exact same itemset extraction logic, identifying both the frequent itemsets of the K th itemset and any additional itemsets that match the support value of the K th itemset. For more details, refer to Algorithm 7.

In lines 3 to 21 of the Algorithm 9, the initial data are traversed again to construct the first level of the BMC-Tree structure based on the current threshold value $minSup$ (line 6). For more details on the construction of the BMC-tree, please refer to Aryabarzan et al. (2018).

5.2.2. Nodesets, negNodesets, FI-Tree construction

Continuing with Algorithm 7, after uncovering the top- k 1-itemsets (line 3), the first-level nodesets are constructed (lines [4–6]). Subsequently, the first-level Frequent Itemsets Tree (*FI_Tree*) is initialized (lines 7–10) and built through an iterative process (lines 11–18). During this iteration, the *Construct_FI_Tree* procedure (line 17) is called (see Algorithm 11), which recursively generates subsequent levels of nodesets / negnodesets and expands the *FI_Tree* while simultaneously raising the minimum support threshold. Ultimately, all detected frequent itemsets are added to the dictionary F .

The *Construct_FI_Tree* function (Algorithm 11) processes node N of the frequent itemset tree, using both the parent's frequent itemsets (FIS_{parent}) and the current minimum support threshold ($minSup$) as input. Dynamically generates supersets from *NodeSets* or *NegNodeSets* for level greater than or equal to two while progressively increasing $minSup$, then recursively evaluates these candidates through depth-first traversal to exhaustively identify all frequent itemsets.

The HTK-negFIN *construct_FI_Tree* algorithm key differences from the original implementation have been highlighted in bold face in the Algorithm 11 and are shown below:

- The parameter $minSup$ now is both an input and output, dynamically updated by the Q-Heap mechanism during the function's operation.
- Line 24 checks if new itemsets meet the increasing $minSup$ threshold - if not, they're discarded.
- Line 25 uses the *QHeap_Insert* function, already used in HTK-Miner and described by (Algorithm 3), to keep the Q-Heap structure and $minSup$ value synchronized.
- Line 32 only builds the next tree levels if node N 's support remains greater than or equal to current $minSup$.

All other algorithm parts remain exactly the same as in the original (Aryabarzan et al., 2018) implementation.

In the final step of the HTK-negFIN algorithm, the *getTopKFI2* function (see Section 5.2.1) is invoked one last time to prune any

Algorithm 11 HTK-negFIN procedure Construct_FI_Tree.

Require: $N, FIS_{parent}, \text{minSup}$
Ensure: F (Updated frequent itemsets), minSup (Updated)

```

1:  $P \leftarrow N.\text{itemset}; N.\text{children\_list} \leftarrow \emptyset; N.\text{equivalent\_items} \leftarrow \emptyset$ 
2: for each item  $i \in L_i \wedge i > N.\text{item\_name}$  do
3:    $R = P \cup \{i\}; R.\text{NegNodeset} \leftarrow \emptyset$ 
4:   if  $N.\text{level} = 1$  then
5:     for each  $N.\text{info ni} \in \text{Nodeset}(P)$  do
6:       if  $ni.\text{bitmap\_code} \wedge 2^{\text{index}(i)} = 0$  then
7:          $R.\text{NegNodeset} \leftarrow R.\text{NegNodeset} \cup \{ni\}$ 
8:       end if
9:     end for
10:  else
11:     $jX \leftarrow P \triangleright j$  is leftmost item in  $P$ ,  $X$  is the remaining itemset
12:     $Q \leftarrow iX$   $\triangleright$  Replace first item in  $P$  with  $i$  ( $i > j$ )
13:    for each  $N.\text{info ni} \in \text{NegNodeset}(Q)$  do
14:      if  $ni.\text{bitmap\_code} \wedge 2^{\text{index}(j)} = 1$  then
15:         $R.\text{NegNodeset} \leftarrow R.\text{NegNodeset} \cup \{ni\}$ 
16:      end if
17:    end for
18:  end if
19:   $R'.\text{support} \leftarrow \sum_{ni \in \text{NegNodeset}(R)} ni.\text{count}$ 
20:   $R.\text{support} \leftarrow P.\text{support} - R'.\text{support}$ 
21:  if  $R.\text{support} = P.\text{support}$  then
22:     $N.\text{equivalent\_items} \leftarrow N.\text{equivalent\_items} \cup \{i\}$ 
23:  else if  $R.\text{support} \geq \text{minSup}$  then
24:     $\text{minSup} \leftarrow \text{QHeap.insert}(R.\text{support})$   $\triangleright$  Algorithm 3
25:     $\text{child}_i.\text{level} \leftarrow N.\text{level} + 1$   $\triangleright$  Create node  $\text{child}_i$ 
26:     $\text{child}_i.\text{item\_name} \leftarrow i; \text{child}_i.\text{itemset} \leftarrow R$ 
27:    Append  $\text{child}_i$  to  $N.\text{children\_list}$ 
28:  end if
29: end for
30: if  $\text{minSup} \leq N.\text{count}$  then
31:   $SS \leftarrow$  all subsets of  $N.\text{equivalent\_items}$ 
32:   $PSet \leftarrow \{A \mid A = \{N.\text{item\_name}\} \cup A', A' \in SS\}$ 
33:  if  $FIS_{parent} = \emptyset$  then  $FIS_N \leftarrow PSet$ 
34:  else  $FIS_N = \{P' \mid P' = P_1 \cup P_2, P_1 \in PSet \wedge P_2 \in FIS_{parent}\}$ 
35:   $F = F \cup FIS_N$ 
36:  if  $N.\text{children\_list} \neq \emptyset$  then
37:    for each  $\text{child}_i \in N.\text{children\_list}$  do
38:       $F = F \cup \text{Construct\_FI\_Tree}(\text{child}_i, FIS_N, \text{minSup})$ 
39:    end for
40:  end if
41: end if

```

non-frequent itemsets and return the definitive set of top- k frequent itemsets.

6. Complexity analysis

In this section, we provide a formal complexity analysis. Our analysis primarily focuses on the **BSN mode** of the **HTK-Miner**, as the experimental results in Section 7.2 identify this configuration as the dominant performer for top- k search space exploration. Regarding **HTK-negFIN**, it utilizes the specialized *NegNodeset* data structure inherited from the original negFIN algorithm rather than the multi-mode representations of HTK-Miner. Because the core mining logic of HTK-negFIN follows the established theoretical framework of negFIN—with modifications restricted to the integration of the Q-Heap for top- k threshold management—its complexity is governed by the same established bounds. Therefore, we omit a separate detailed analysis for HTK-negFIN to maintain conciseness and focus on the unique advantages of the HTK-Miner (BSN).

6.1. Mining process complexity

The total time complexity of the mining phase in BSN mode is a function of the number of candidate itemsets evaluated (N_{cand}) and the cost of the bitwise operations. For each candidate, a bitwise AND is performed on BSNs, requiring $O(\lceil |D|/w \rceil)$ time, where w is the processor word size (e.g. 64 bit).

The overall complexity for the mining process in BSN mode is defined as:

$$T_{mining} = O(N_{cand} \cdot \frac{|D|}{w} + N_{ins} \cdot (\log k + k)) \quad (1)$$

where N_{ins} represents the number of itemsets inserted into the Q-Heap. In practice, the dynamic pruning and complexity advantages of the Q-Heap detailed in Section 4.4.1, combined with the SIME-driven *minSup* acceleration and optimizations in Section 4.6, ensure that N_{cand} remains significantly lower than the $O(2^n)$ theoretical search space. This reduction in candidate generation, combined with the $O(|D|/w)$ efficiency of BSN, provides the theoretical basis for the performance gains observed against BTK and other state-of-the-art algorithms.

7. Experimental results and analysis

We conducted two groups of experiments. In the first group, the performance of the four modes of HTK-Miner - VR with TS, BSN, DTS and DBSN (see Section 4.5) – is compared in terms of both runtime and memory consumption. In the second set of experiments, HTK-Miner and HTK-negFIN are compared against each other, as well as against BTK (Dam et al., 2016). To evaluate their overall performance, we also included negFIN (Aryabarzan et al., 2018), which requires a minimum support threshold instead of the Top- k parameter. All the algorithms tested recovered the exact same frequent itemsets, but differed significantly in terms of runtime and memory consumption.

7.1. Datasets and environment

We utilized ten transactional datasets, eight of which are real world datasets, and the remaining two are synthetic. Seven of the real world datasets, *Kosarak*, *Pumsb*, *Accidents*, *Mushroom*, *Connect*, *Chess*, and *Webdocs*, are widely recognized and are sourced from diverse domains such as market basket analysis, census data, biology, statistics and world wide web. These datasets are highly popular, frequently used in previous studies, and can be downloaded from the FIMI repository (Goethals and Zaki, 2003).

We developed the L-0023 dataset. It contains one year of sales records from a commercial cosmetics business in Greece. Originally a *INV* type dataset (Malliaridis et al., 2024), it was transformed into a *MBL* transactional database format for use in subsequent experiments.

The remaining datasets are synthetic. The T40I10D100K dataset, also downloaded from the FIMI repository, is a sparse dataset with 100,000 transactions, 943 distinct items, and an average of 40 items per transaction. In contrast, we generated the T16IT20D100K dataset by using the synthetic data generation tool from the Scikitlearn itemset mining library (scikit-mine developers, 2025). T16IT20D100K is a highly dense dataset. It consists of 100,000 transactions, 20 distinct items and an average of 16.1 items per transaction, resulting in a density of 80 percent.

All these datasets, as presented in Table 9, cover a wide spectrum of categories, ranging from very sparse to very dense (density from 0.00003 to 0.8), from datasets with few transactions to those with millions of transactions (3,196 to 1,692,082) and from datasets with few items to those with many items (20 to 5,267,656), as well as combinations of these characteristics.

Notably, the inclusion of the *webdocs* dataset, featuring over five million unique items, serves as a significant large-scale benchmark that stresses the limits of FIM algorithms in terms of memory management and computational efficiency. Since complexity in itemset mining is

Table 9

Dataset characteristics in sparse to dense order.

Dataset	Source/type	Trans	Items	Trans avg size	Density
webdocs	FIMI, real	1,692,082	5,267,656	177.2	0.00003
kosarak	FIMI, real	990,002	41,270	8.1	0.00019
L-0023	self, real	3587	2160	18.1	0.00839
pumsb	FIMI, real	49,046	2113	74.0	0.03502
T40I10D100K	FIMI, synth	100,000	943	40.6	0.04306
accidents	FIMI, real	340,183	469	34.8	0.07422
mushroom	FIMI, real	8124	119	23.0	0.12328
connect	FIMI, real	67,557	129	43.0	0.33333
chess	FIMI, real	3196	75	37.0	0.49333
T16IT20D100K	self, synth	100,000	20	16.1	0.80584

fundamentally driven by the combinatorial search space—which is dictated by the density, the number of unique items, and the number of transactions—this diverse selection provides a comprehensive stress test for the proposed algorithms generalization ability, ensuring their performance is evaluated across both sparse real-world logs and highly dense environments such as census or biological data.

All experiments were carried out on a system equipped with an Intel Core i5-10400 processor (6 cores, 12 threads), 16 GB of RAM, and the Windows 11 Professional operating system.

The HTK-Miner, HTK-negFIN, and BTK algorithms were implemented in Python and can be downloaded from GitHub.¹ The original negFIN algorithm, also implemented in Python, was downloaded from its creator's GitHub repository.²

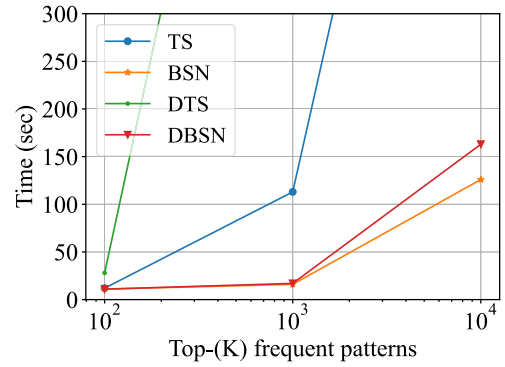
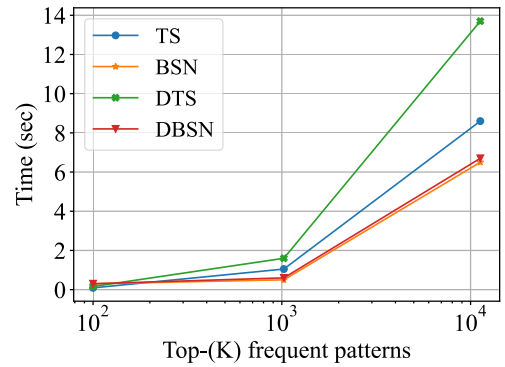
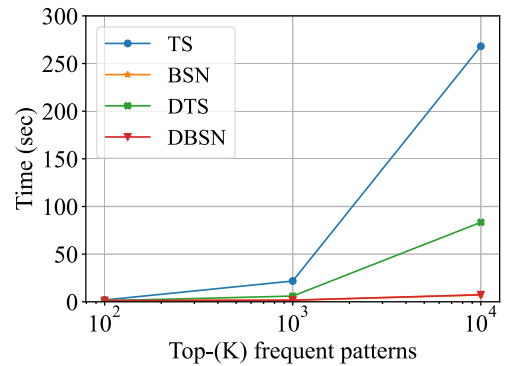
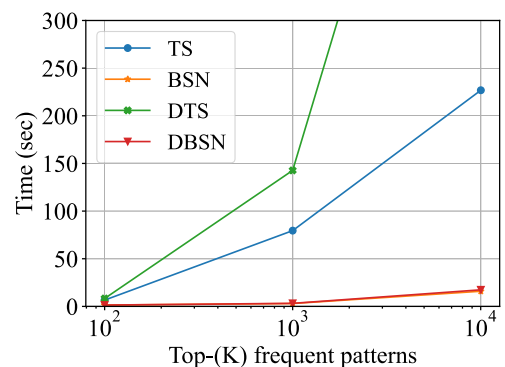
7.2. Evaluating the HTK-Miner's modes

This section evaluates the four vertical data representations employed by HTK-Miner: VR with TS, BSN, DTS, and DBSN as introduced in Section 4.5. These experiments are crucial in determining which of the four modes performs better, in both runtime and memory manipulation factors, than the others and under what conditions. The key question is whether there is a mode that is universally superior to the others or if specific modes perform better on sparse or dense data, datasets with many or few items, or datasets with many or few transactions.

In the first group of figures, the performance of runtime is evaluated, while in the second group, the level of memory management is assessed. Furthermore, the results are presented in order from the most sparse to the most dense datasets as shown in Table 9, with the exception of the *webdocs* dataset, which is presented last due to its exceptionally large size and the unique handling it required. In all experiments, the total execution time of the algorithm was measured for each mode, which includes the time taken to read the dataset, convert it into the appropriate representation format depending on the mode (TS, BSN, DTS, DBSN), execute the algorithm to find the Top- k frequent items, and perform statistical calculations for execution time and memory consumption measurements. In all experiments, the values 100, 1,000, and 10,000 were used for the parameter k , with execution constrained by thresholds of 1000 seconds for runtime and 12 GB for memory usage.

7.2.1. Runtime performance

The runtime results are presented in seconds. The figures are capped at a maximum display of 300 s, except Fig. 10 which displays up to 1000 s, allowing the lines to indicate trends for values exceeding this time limit. In Figs. 1–9, it was observed that the two implementations using bitsets (BSN, DBSN) outperformed their counterparts based on tidlist representations (TS, DTS). Specifically, they emerged as the fastest solutions globally for larger values of k (1,000 and 10,000) by one to two orders of magnitude.

**Fig. 1.** Runtime on Kosarak.**Fig. 2.** Runtime on L-0023.**Fig. 3.** Runtime on Pumsb.**Fig. 4.** Runtime on T40I10D100K.

¹ <https://github.com/terminal0gr/HTKMiner>

² <https://github.com/aryabarzan/negFINpy>

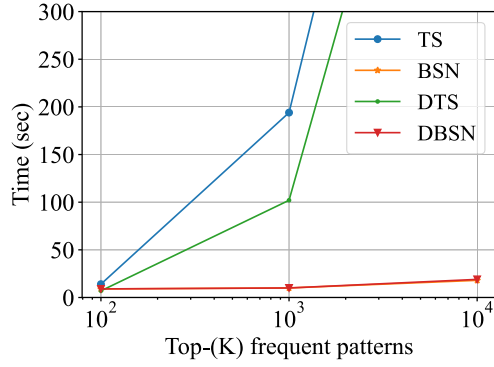


Fig. 5. Runtime on Accidents.

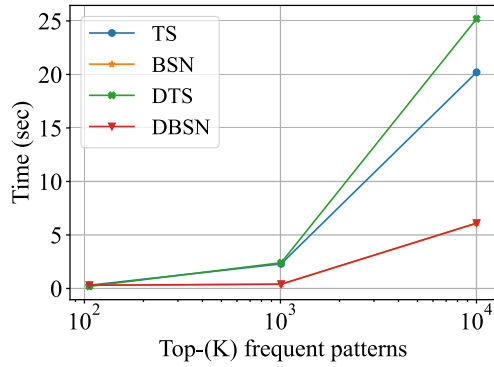


Fig. 6. Runtime on Mushroom.

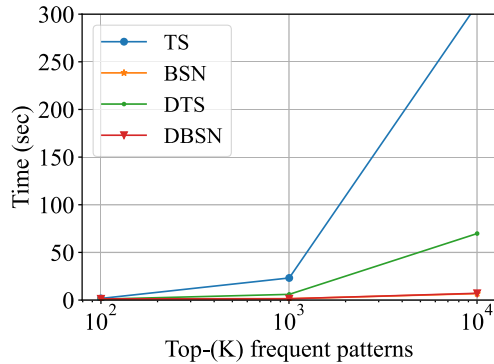


Fig. 7. Runtime on Connect.

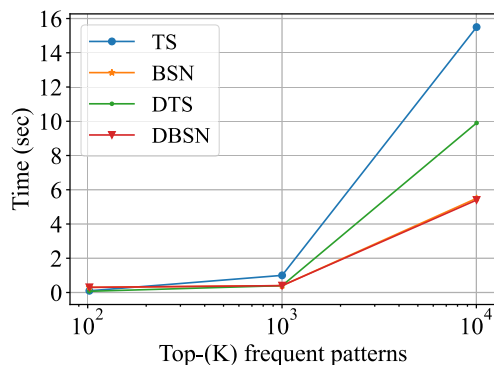


Fig. 8. Runtime on Chess.

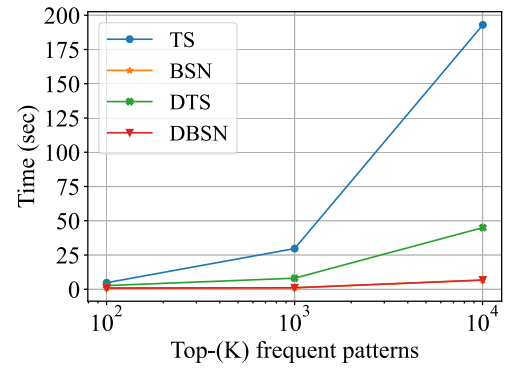


Fig. 9. Runtime on T16IT20D100K.

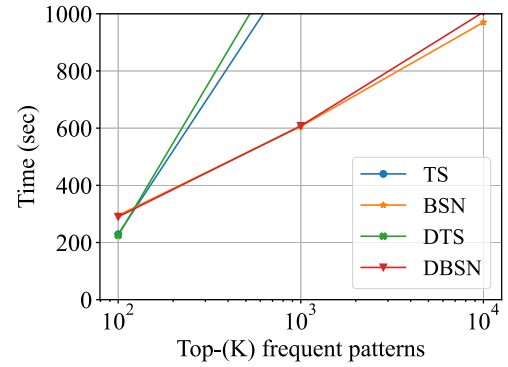


Fig. 10. Runtime on Webdocs.

For $K = 100$, the results were more varied. For instance, in the chess dataset shown in Fig. 8, the DTS representation was the fastest (0.07s), followed by TS (0.09s), and then BSN and DBSN (0.29s). We concluded that for a small k threshold, the TS and DTS modes are competitive with the corresponding BSN and DBSN modes. This is because maintaining fewer Top- k frequent itemsets and using small to medium sized data allows for a much faster increase in the dynamic minimum support, thereby requiring fewer intersection operations between tidsets or diffsets, which are slower compared to operations between bitsets. Additionally, in the BSN and DBSN modes, the time required for converting tidsets to bitsets during the algorithm's initial stage is also taken into account.

For higher values of K , it is evident that the bitset modes are superior and more scalable. Specifically, the BSN and DBSN modes demonstrate a greater efficiency 55% to 97% (see Figs. 1 to 10).

In non-bitset modes, TS outperforms DTS on sparse data, while the opposite is true for dense data with a density of approximately 0.2 and above.

In the case of the *webdocs* dataset, the experiments yielded slightly different results. With the k parameter set to 100, the TS and DTS modes are clearly more efficient, followed by BSN and DBSN. This happens because the specific dataset contains approximately 1.7 million transactions, which impacted the conversion to bitset, negatively impacting the performance of the BSN and DBSN modes. For k values of 1000 and above, the TS and DTS modes were unable to complete the mining process within the time limit of 1000 s. In these cases, the BSN mode outperformed the corresponding DBSN mode as shown in Fig. 10.

BSN consistently outperforms DBSN across all evaluated cases, including dense datasets. This occurs because, in DBSN mode, two bitwise operations are required to compute the difference between bitset numbers, whereas only one bitwise operation is needed in BSN mode. In general, the BSN mode is more efficient than the other modes in almost all scenarios.

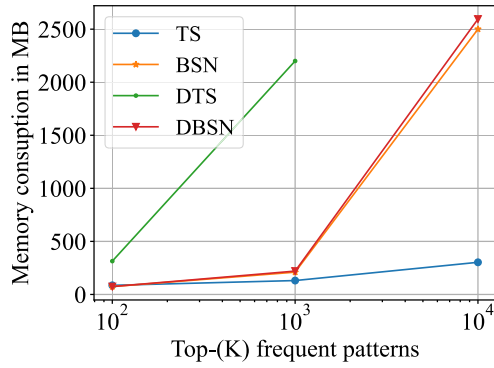


Fig. 11. Memory usage on Kosarak.

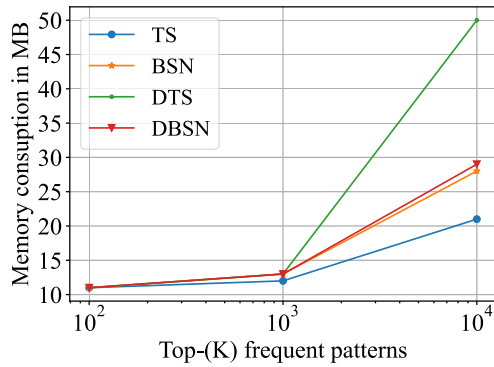


Fig. 12. Memory usage on L-0023.

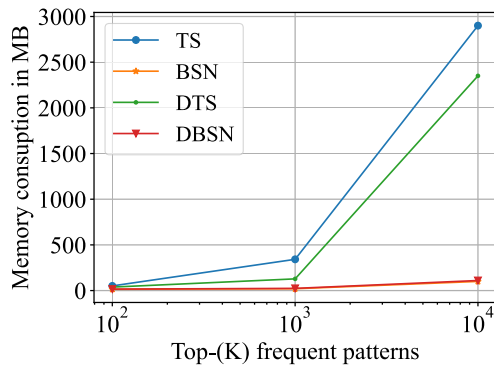


Fig. 13. Memory usage on Pumsb.

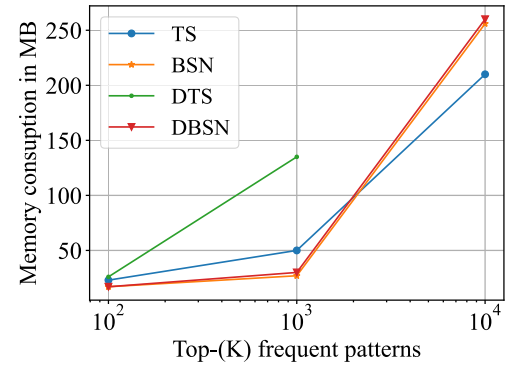


Fig. 14. Memory usage T40I10D100K.

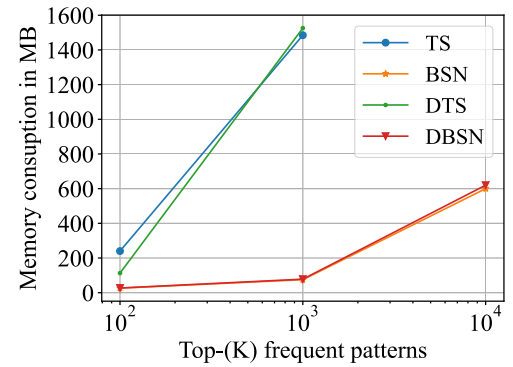


Fig. 15. Memory usage on Accidents.

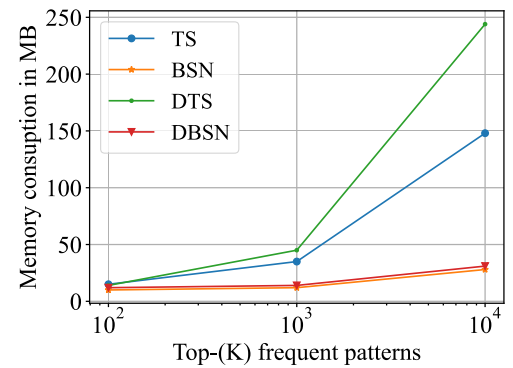


Fig. 16. Memory usage on Mushroom.

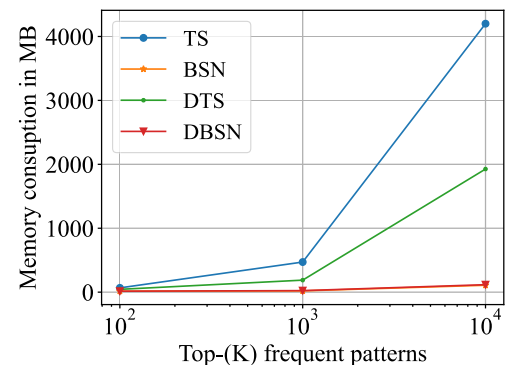


Fig. 17. Memory usage on Connect.

7.2.2. Memory consumption results

The memory consumption results are presented in megabytes (MB) and encompass initial data reading, frequent itemset mining and the statistical analysis of runtime and memory consumption. For measuring memory, Unique Set Size (USS) Memory was used, which measures the unique memory consumption of a process or application. It represents the actual amount of physical memory that is exclusively used by a process (see Figs. 11–20).

In general, memory management is considered efficient, as evidenced by the improvements made to HTK-Miner, which are analyzed in Section 4.6. Key enhancements include the use of a bidirectional dictionary, the adoption of simple data structures for storage, the storage of only the current level during mining, the implementation with iteration methods instead of recursion, the use of the Smart Iteration MEchanism (SIME) and other programming techniques, resulting in an overall memory savings of 2 to 3 orders of magnitude.

When comparing the four modes, it was observed that the bitset modes (BSN, DBSN) exhibit better memory management, and in no

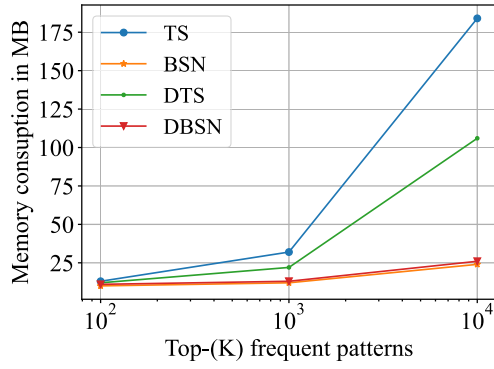


Fig. 18. Memory usage on Chess.

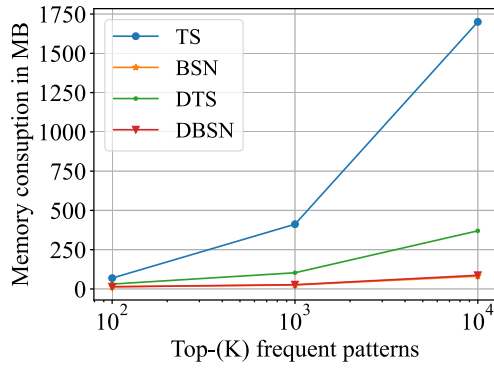


Fig. 19. Memory usage T16IT20D100K.

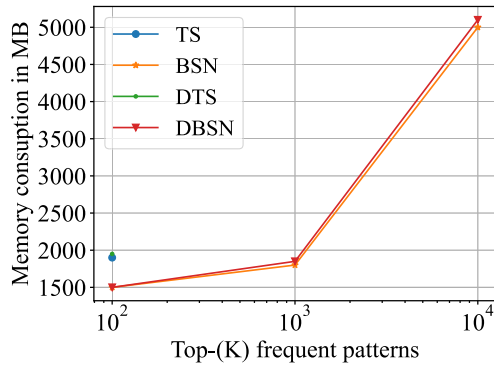


Fig. 20. Memory usage on Webdocs.

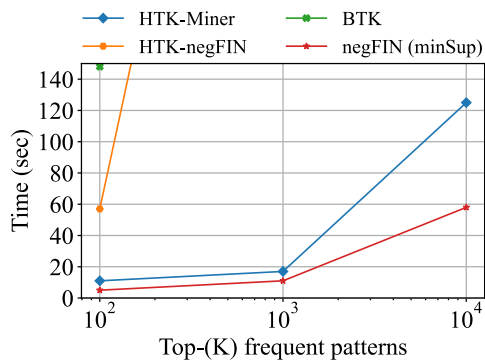


Fig. 21. Runtime on Kosarak.

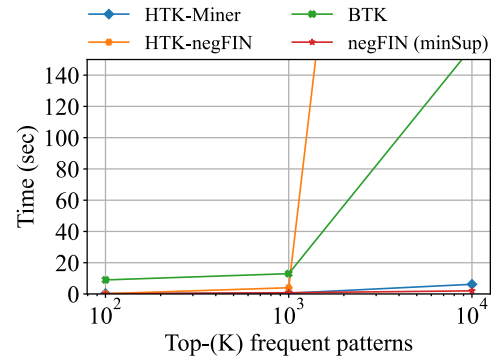


Fig. 22. Runtime on L-0023.

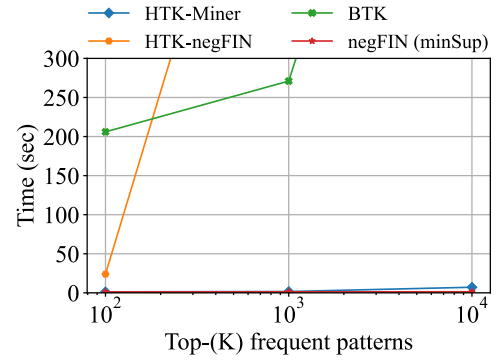


Fig. 23. Runtime on Pumsb.

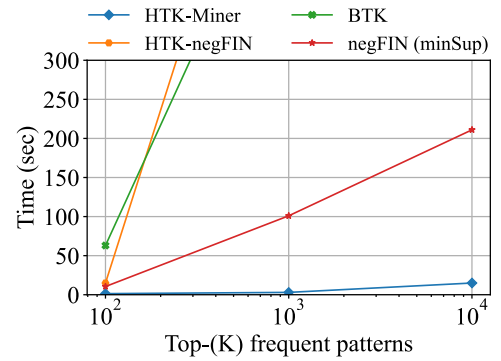


Fig. 24. Runtime on T40I10D100K.

experiment did they exceed 5 GB. The TS mode appears to have superior memory management for very sparse datasets (*kosarak*, *L* – 0023, see Figs. 31,12), particularly for large values of k . The use of diffsets (DTS) is more memory efficient for dense datasets, as expected, although it does not outperform the corresponding bitset implementations in terms of memory saving (Figs. 17–19). In specific experiments with high values of k (e.g. 10,000), memory consumption was not recorded for the DTS and TS modes because the time limit of 1000 s was exceeded. Finally, a direct comparison of the two bitset modes (BSN and DBSN) concluded that they have almost identical memory management.

In summary, the BSN mode is deemed the most efficient based on the experimental results, both in terms of execution time and memory consumption, in almost all cases, closely followed by the corresponding DBSN mode. The use of tidsets (TS) is more time-efficient only for small values of k (e.g. 100) in sparse datasets with a very large number of transactions (see *webdocs* Fig. 10). For this reason, in the comparative experiments with other algorithms presented in Section 7.3, the HTK-Miner will be represented by its BSN mode.

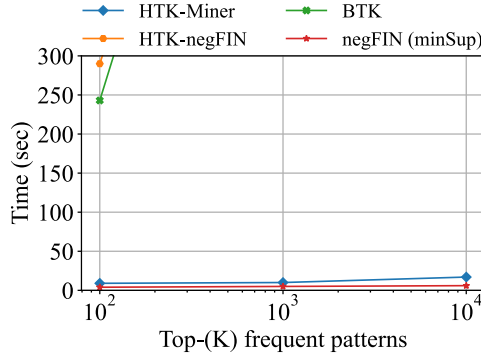


Fig. 25. Runtime on Accidents.

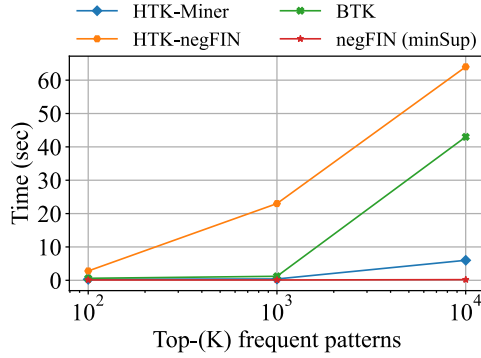


Fig. 26. Runtime on Mushroom.

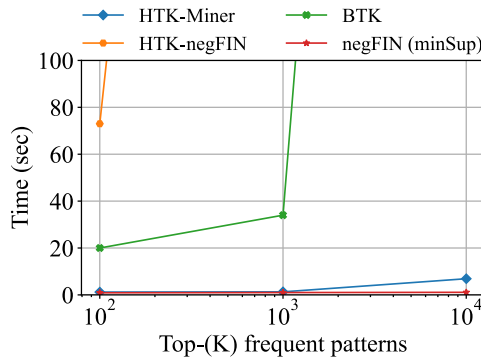


Fig. 27. Runtime on Connect.

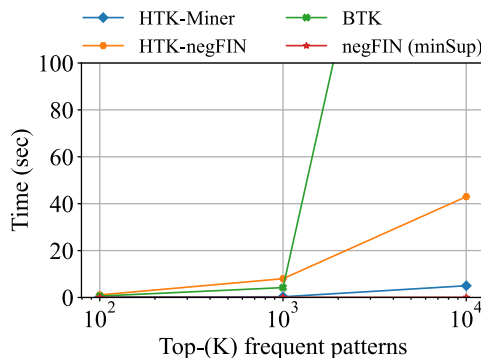


Fig. 28. Runtime on Chess.

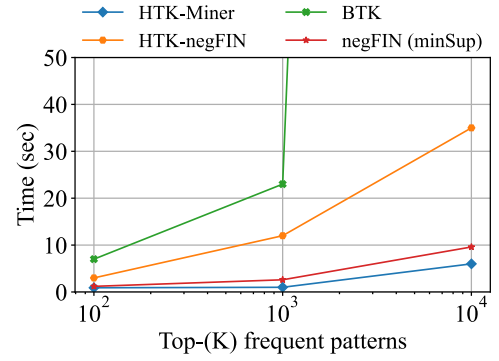


Fig. 29. Runtime on T16IT20D100K.

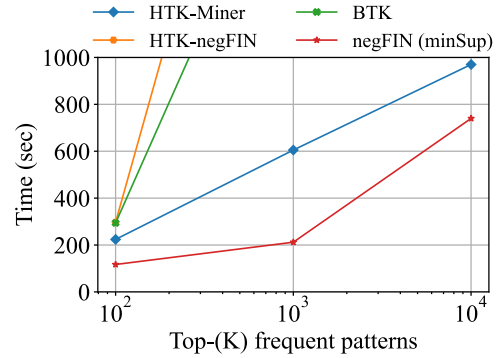


Fig. 30. Runtime on Webdocs.

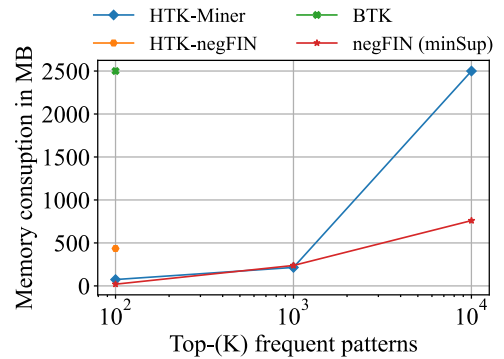


Fig. 31. Memory usage on Kosarak.

7.3. HTK-Miner and HTK-negFIN experimental study

HTK-Miner and HTK-negFIN were evaluated both against each other and against BTK (Dam et al., 2016), a state-of-the-art top- k ranking algorithm. To ensure a rigorous evaluation, the algorithms are compared with the original negFIN implementation (Aryabarzan et al., 2018), which serves as an idealized performance benchmark. While negFIN is provided with the optimal *minSup* (pre-calculated), our algorithms discover the threshold dynamically. The results indicate that HTK-Miner often matches or exceeds this 'idealized' benchmark, particularly in high values of k , validating the efficiency of our BSN-based mining logic.

All algorithms were implemented in Python. HTK-Miner, HTK-negFIN, and BTK were developed in this study and are available as open-source projects on GitHub.³ The negFIN algorithm, also implemented in Python, was obtained directly from its creator's GitHub

³ <https://github.com/terminal0gr/HTKMiner>

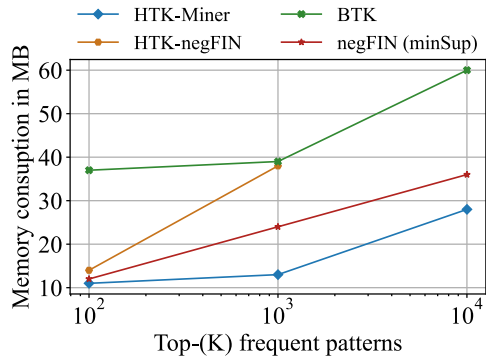


Fig. 32. Memory usage on L-0023.

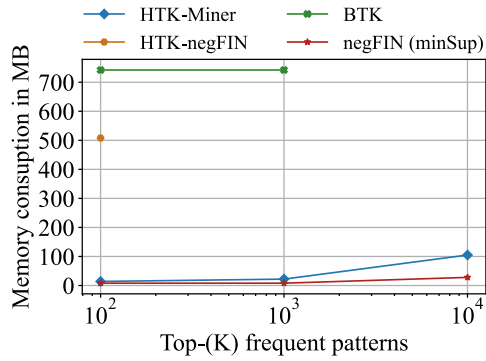


Fig. 33. Memory usage on Pumsb.

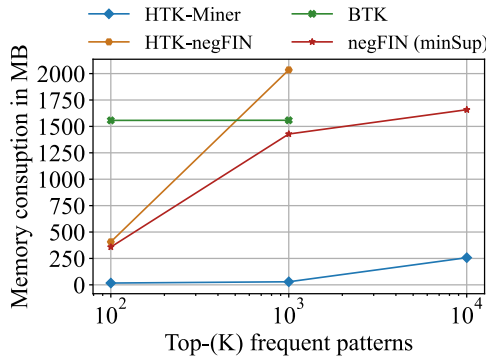


Fig. 34. Memory usage T40I10D100K.

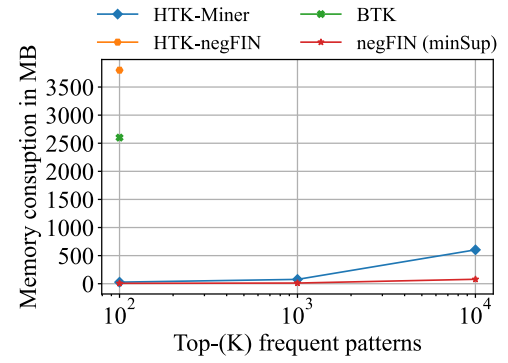


Fig. 35. Memory usage on Accidents.

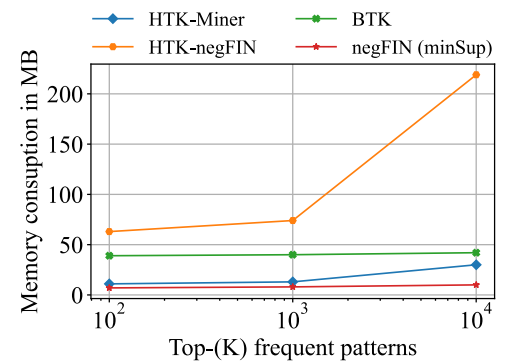


Fig. 36. Memory usage on Mushroom.

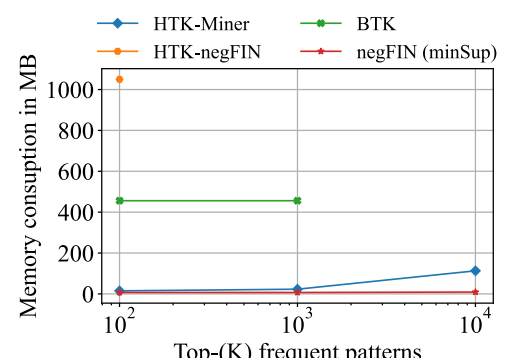


Fig. 37. Memory usage on Connect.

repository.⁴ All experiments were carried out in the same hardware (see Section 7.1)

In all experiments, the values of 100, 1,000, and 10,000 were used for the k parameter. The optimal Top- k ranking and minimum support threshold parameters for the BTK and negFIN algorithms were determined based on statistical results obtained from HTK-Miner's execution. The BSN mode of HTK-Miner was employed, as it demonstrated the highest efficiency based on the experiments presented in Section 7.2, throughout this experimental series. Experiments were conducted with thresholds of 1000 s for runtime and 12GB for memory usage. Although all algorithms tested identified the **exact same** frequent itemsets, they exhibited significant differences in runtime and memory consumption.

7.3.1. Runtime results

Runtime results, measured in seconds, include all processing stages: initial data reading, frequent itemset mining, and statistical analysis run-

time across the ten datasets described in Section 7.1. The X axis represents values of k and the Y axis stands for the runtime in seconds. Figs. 21–30 present these results with adaptive display caps ranging from 60 to 1000 s, dynamically scaled based on observed runtime distributions, while still showing performance trends beyond this limit. During the experiments, certain algorithms failed to produce results within the specified thresholds. In these cases, their symbols appear in the top-left corner of the graph for visualization purposes only. The results follow a progression from sparsity to density as per Table 9, with *WebDocs* treated as a special case although sparse, due to its exceptional size (1.6 million records, 5.2 million items) and unique processing requirements, which justify its final position in the presentation (Fig. 30).

To analyze the time and memory requirements of each algorithm, we outline their key components and data structures. The BTK method utilizes a TB-Tree with B-lists and subsume indexes of frequent 1-itemsets (Dam et al., 2016), employing RSC and subsume strategies to minimize candidate itemsets, while negFIN (Aryabarzan et al., 2018) and HTK-negFIN algorithms adopt a more compressed BMC-Tree structure with Nodesets/NegNodeSets instead of B-lists, along with a frequent itemset

⁴ <https://github.com/aryabarzan/negFINpy>

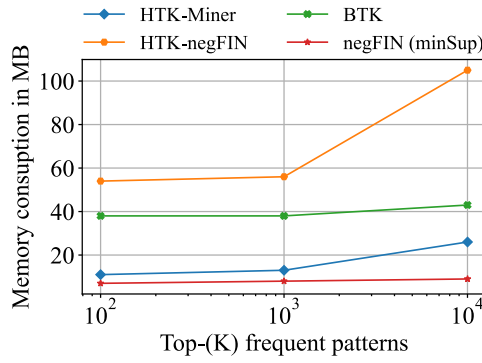


Fig. 38. Memory usage on Chess.

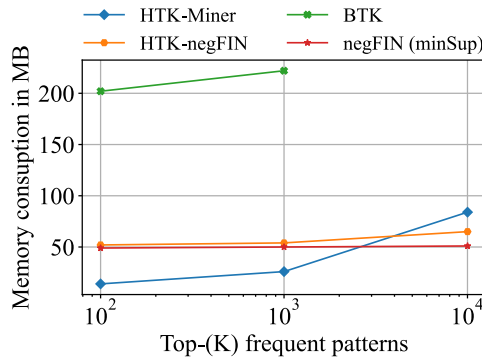


Fig. 39. Memory usage T16IT20D100K.

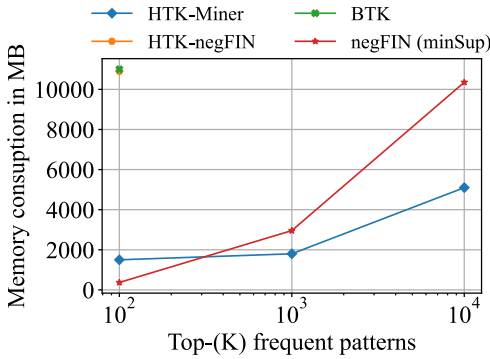


Fig. 40. Memory usage on Webdocs.

tree for itemset mining - with HTK-negFIN additionally incorporating a Q-Heap structure for dynamic minimum support adjustment. Similarly, HTK-Miner's BSN mode implements a vertical bitset data representation and a Q-Heap structure for efficient support threshold management, combined with SIME and other advanced pruning techniques to optimize search space exploration and reduce candidate generation (see Section 4.6).

The experimental results show that, across all tested values of k and datasets, HTK-Miner consistently outperforms all other top- k itemset mining algorithms (HTK-negFIN, BTK), achieving efficiency gains ranging from $3\times$ to over $200\times$ compared to competing methods. This performance advantage is particularly evident in the T40I10D100K dataset with $K = 10^3$, where HTK-Miner completes in just 3 s compared to HTK-negFIN's 750 s and BTK's 572 s (Fig. 24), highlighting its remarkable scalability and optimization effectiveness. The major reason for this performance difference lies in the fundamental algorithmic approaches: both BTK and HTK-negFIN must construct nearly complete tree structures (TB-Tree and BMC-Tree respectively) during initial execution since no minimum support threshold is predefined. This becomes particu-

larly computationally expensive when k -values approach or exceed the dataset's item count, as their search space pruning mechanisms only activate after this exhaustive construction phase. In contrast, HTK-Miner's initial overhead is limited to creating its vertical bitset representation - a process significantly accelerated through parallel processing techniques, giving it a substantial efficiency advantage from the outset.

Another key observation is that while HTK-Miner and negFIN successfully produced results for all k -values, ranging from 100 to 10,000, within the specified runtime (< 1000 s) and memory limits (< 12 GB), BTK and HTK-negFIN failed to complete execution, particularly at larger k -values, due to exceeding the runtime threshold.

The two top- k tree-based approaches (BTK and HTK-negFIN) demonstrate comparable efficiency in itemset mining, as both rely on similar hierarchical structures (TB-Tree and BMC-Tree, respectively) and face identical scaling limitations at high k -values. At low k values, HTK-negFIN demonstrates an efficiency advantage, outperforming BTK in 6 out of 10 experimental cases, likely due to its optimized BMC-Tree structure and dynamic Q-Heap support adjustment. However, at high K -values, their performance diverges based on dataset characteristics. BTK excels in datasets with a large number of distinct items (benefiting from its TB-Tree's subsume indexing) while HTK-negFIN prevails in datasets with high transaction counts but fewer unique items. This suggests that item cardinality, rather than dataset density alone, becomes the dominant factor in their face-to-face comparative scalability.

In direct comparison between HTK-Miner and the state-of-the-art negFIN algorithm, negFIN demonstrates greater overall efficiency. However, it is important to note that negFIN was given the optimal minimum support threshold value (calculated from HTK-Miner's statistics) - an advantage unavailable in real-world scenarios, where this typically requires extensive trial-and-error estimation. In addition, their performance differences are minimal and comparable, particularly at lower k -values. Interestingly, HTK-Miner outperformed negFIN in both synthetic datasets, achieving an order of magnitude speed advantage in all k values in the T40I10D100K and T16IT20D100K datasets (Figs. 24,29). These observations suggest that HTK-Miner could potentially be considered as an efficient approach for top- k frequent itemset mining since it offers benefits in both execution time and memory efficiency.

7.3.2. Memory consumption results

The results of memory consumption are presented in megabytes (MB) and encompass initial data reading, frequent itemset mining, statistical analysis of runtime, and memory consumption in the ten datasets described in Section 7.1. The X axis represents values of k while the Y axis stands for the memory consumption in megabytes (MB). For measuring memory, Unique Set Size (USS) Memory was used, which measures the unique memory consumption of a process or application. It represents the actual amount of physical memory that is exclusively used by a process (see Figs. 31–40).

All algorithms employ memory optimization techniques: BTK utilizes the subsume technique (Dam et al., 2016), negFIN and HTK-negFIN leverage the bitset-based BMC-Tree structure (Aryabarzan et al., 2018), while HTK-Miner implements a vertical BSN representation that stores only the current level's data required for deriving the next level's frequent itemsets - progressively releasing memory from previous levels (Section 4). Crucially, none of the algorithms reached the 12GB memory limit in experiments where they completed execution within the designated time constraints.

Among the tested algorithms, BTK demonstrates the best memory management scalability, despite being consistently outperformed by both HTK-Miner and negFIN in all experiments. When comparing BTK and HTK-negFIN, their performance varies: HTK-negFIN proves more efficient at lower k values and with sparse data, while BTK excels with higher k values and dense datasets. Note that these algorithms failed to produce results in cases where they exceeded the 1000 s runtime limit

(indicated by truncated lines or single spots in the figures), preventing memory consumption measurements in those scenarios.

It's particularly noteworthy that while negFIN and HTK-negFIN employ identical methodologies and structures, negFIN, which operates with a fixed and predefined minimum support threshold, consistently outperforms HTK-negFIN, often by over two orders of magnitude in specific cases (e.g., in the Accidents dataset with $K = 100$, as shown in Fig. 35). This demonstrates that although negFIN remains the state-of-the-art among minimum-support-based algorithms, its efficiency diminishes when adapted for top- k mining.

The memory efficiency comparison between negFIN and HTK-Miner yields mixed results. For smaller k values, both algorithms demonstrate comparable performance, while at higher k values, negFIN generally shows superior scalability based on trend-line analysis. However, Fig. 34 reveals HTK-Miner's consistently better memory management, outperforming negFIN by one to two orders of magnitude in some cases. Notably, in Fig. 40 at high k values (e.g. $K = 10^4$), HTK-Miner uses half the memory (5GB vs. 10GB). Overall, HTK-Miner proves equally memory-efficient as negFIN, establishing itself as a robust solution for memory-conscious implementations.

8. Conclusion

This paper addressed the challenge of efficiently discovering the Top- k most frequent itemsets by introducing two algorithms: HTK-Miner and HTK-negFIN. HTK-Miner is rooted in the equivalence class theory and a breadth-first candidate generation strategy, leveraging vertical data structures. Its core strength lies in its flexibility, offering four operational modes (TS, BSN, DTS, and DBSN) and incorporating a suite of powerful optimizations, including the Q-Heap for rapid support estimation, Smart Iteration Mechanism (SIME), and single-pass database reading. HTK-negFIN, conversely, is derived from the pattern-growth family, adapting the efficient negFIN algorithm to the Top- k framework and integrating the beneficial Q-Heap structure.

The experimental evaluation provided compelling evidence of the solutions' efficacy. The superior performance of the BitSet Numbers (BSN) mode is primarily attributed to its computational efficiency, where bitwise AND operations exploit word-level parallelism to minimize the cost of support counting. Furthermore, the combination of the Q-Heap and SIME mechanism facilitates rapid threshold elevation, inducing an aggressive early pruning effect that drastically reduces the number of candidate itemsets compared to existing methods. In a head-to-head comparison with the state-of-the-art Top- k rank algorithm BTK, HTK-Miner (BSN) established itself as the new benchmark, achieving performance gains of up to two orders of magnitude. Furthermore, it maintained comparable execution speed and memory efficiency against the highly-optimized minimum support-based algorithm, negFIN. HTK-negFIN also proved to be a competitive contender, exhibiting performance comparable to BTK.

These findings decisively demonstrate the effectiveness, scalability, and efficiency of the proposed HTK-Miner algorithm for practical Top- k frequent itemset mining.

A key future direction involves integrating HTK-Miner into an AutoML framework for Association Rule Mining (ARM). This addresses the main barrier for non-expert users: specifying the complex minimum support threshold. Utilizing HTK-Miner's efficiency and its Quick Heap structure, the AutoML tool will allow users to simply input the desired number of rules (k). HTK-Miner will dynamically calculate the necessary support, abstracting the parameter tuning entirely. The system will then automatically generate and rank the final association rules based on multiple interestingness metrics. This integration will create a scalable and truly threshold-free ARM solution, enabling non-experts to effortlessly and effectively mine valuable patterns.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used AI tools in order to make it more concise and formal. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the publication.

CRedit authorship contribution statement

Konstantinos Malliaridis: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization; **Stefanos Ougiaroglou:** Conceptualization, Methodology, Validation, Formal analysis, Writing – review & editing, Supervision, Project administration.

Data availability

The datasets and source code associated with this study are publicly available in the following GitHub repository: <https://github.com/terminal0gr/HTKMiner>.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The publication of the article in OA mode was financially supported by HEAL-Link.

References

- Abdelaal, A. A., Al-Shayegi, M., Allaho, M. et al. (2021). Customized frequent patterns mining algorithms for enhanced top-rank- k frequent pattern mining. *Expert Systems with Applications*, 169, 114530.
- Aggarwal, C. C., Bhuiyan, M. A., & Hasan, M. A. (2014). Frequent pattern mining algorithms: A survey. In C. C. Aggarwal, & J. Han (Eds.), *Frequent pattern mining* (pp. 19–64). Cham: Springer International Publishing. https://doi.org/10.1007/978-3-319-07821-2_2
- Agrawal, R., Imieliński, T., & Swami, A. (1993). Mining association rules between sets of items in large databases. In *Proceedings of the 1993 ACM SIGMOD international conference on management of data* (pp. 207–216).
- Aryabarzan, N., Minaei-Bidgoli, B., & Teshnehlab, M. (2018). negFIN: An efficient algorithm for fast mining frequent itemsets. *Expert Systems with Applications*, 105, 129–143.
- Ashraf, J., Habib, A., & Salam, A. (2016). Top- k miner: Top- k identical frequent itemsets discovery without user support threshold. *Knowledge and Information Systems*, 48(3), 741–762.
- B. Goethals, Frequent itemset mining implementations 2004, *CEUR Workshop Proceedings* 126 (2004). <https://ceur-ws.org/Vol-126/>.
- B. Goethals, M. Zaki, Frequent itemset mining dataset repository, 2003. <http://fimi.uantwerpen.be/data/>.
- B. Goethals, Workshop on frequent itemset mining implementations (fimi'03), 2003. <http://fimi.uantwerpen.be/fimi03/>.
- Bodan, F. (2003). A fast APRIORI implementation. In B. Goethals, & M. J. Zaki (Eds.), *FIMI'03, Frequent itemset mining implementations, proceedings of the ICDM 2003 workshop on frequent itemset mining implementations, 19 December 2003, Melbourne, Florida, USA* (pp. 1–10). (vol. 90). CEUR Workshop Proceedings. <https://ceur-ws.org/Vol-90/bodan.pdf>.
- Chon, K.-W., & Kim, C. (2024). Gminer + +: Boosting GPU-based frequent itemset mining by reducing redundant computations. *Expert Systems With Applications*, 250, 123928.
- Chuang, K.-T., Huang, J.-L., & Chen, M.-S. (2008). Mining top- k frequent patterns in the presence of the memory constraint. *The VLDB Journal*, 17, 1321–1344.
- Czubryt T. J., Leung C. K., Pazdor A. G., Q-viper: Quantitative vertical bitwise algorithm to mine frequent patterns, in: *International conference on big data analytics and knowledge discovery*, Springer, 2022, pp. 219–233.
- Dam, T.-L., Li, K., Fournier-Viger, P., & Duong, Q.-H. (2016). An efficient algorithm for mining top-rank- k frequent patterns. *Applied Intelligence*, 45, 96–111.
- Deng, Z., Wang, Z., & Jiang, J. (2012). A new algorithm for fast mining frequent itemsets using n-lists. *Science China Information Sciences*, 55, 2008–2030.
- Deng, Z.-H. (2014). Fast mining top-rank- k frequent patterns by using node-lists. *Expert Systems with Applications*, 41(4), 1763–1768.

- Deng, Z.-H. (2016). Diffnodesets: An efficient structure for fast mining frequent itemsets. *Applied Soft Computing*, 41, 214–223.
- Deng, Z.-H., & Fang, G.-D. (2007). Mining top-rank-k frequent patterns. In *2007 International conference on machine learning and cybernetics* (pp. 851–856). IEEE (vol. 2).
- Deng, Z.-H., & Lv, S.-L. (2014). Fast mining frequent itemsets using nodesets. *Expert Systems with Applications*, 41(10), 4505–4512.
- Deng, Z.-H., & Lv, S.-L. (2015). Prepost+: An efficient n-lists-based algorithm for mining frequent itemsets via children–parent equivalence pruning. *Expert Systems with Applications*, 42(13), 5424–5432.
- Dong, J., & Han, M. (2007). BittableFI: An efficient mining frequent itemsets algorithm. *Knowledge-Based Systems*, 20(4), 329–335.
- Fang, G.-D., & Deng, Z.-H. (2008). VTK: Vertical mining of top-rank-k frequent patterns. In *2008 Fifth international conference on fuzzy systems and knowledge discovery* (pp. 620–624). IEEE (vol. 2).
- Fournier-Viger, P., Lin, J. C.-W., Vo, B., Chi, T. T., Zhang, J., & Le, H. B. (2017). A survey of itemset mining. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 7(4), e1207.
- Fournier-Viger, P., Wu, C.-W., Zida, S., & Tseng, V. S. (2014). FHM: Faster high-utility itemset mining using estimated utility co-occurrence pruning. In *Foundations of intelligent systems: 21st international symposium, ISMIS 2014, Roskilde, Denmark, June 25–27, 2014. proceedings 21* (pp. 83–92). Springer.
- Grahne, G., & Zhu, J. (2005). Fast algorithms for frequent itemset mining using fp-trees. *IEEE Transactions on Knowledge and Data Engineering*, 17(10), 1347–1362.
- Gündüz, Ş., & Özsü, M. T. (2003). A web page prediction model based on click-stream tree representation of user behavior. In *Proceedings of the ninth ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 535–540).
- Guo, S.-B., Liu, D.-Y., Fang, X.-J., Meng, Y., Zhou, Z.-Z., Li, J., Li, M., Luo, L.-L., Li, H.-L., Cai, X.-Y. et al. (2025). Current concerns and future directions of large language model chatGPT in medicine: A machine-learning-driven global-scale bibliometric analysis. *International Journal of Surgery*, (pp. 10–1097). <https://doi.org/10.1097/JS9.0000000000003668>
- Han, J., Cheng, H., Xin, D., & Yan, X. (2007). Frequent pattern mining: Current status and future directions. *Data Mining and Knowledge Discovery*, 15(1), 55–86.
- Han, J., Pei, J., & Yin, Y. (2000). Mining frequent patterns without candidate generation. *ACM Sigmod Record*, 29(2), 1–12.
- Han, J., Wang, J., Lu, Y., & Tzvetkov, P. (2002). Mining top-k frequent closed patterns without minimum support. In *2002 IEEE International conference on data mining, 2002. proceedings*. (pp. 211–218). IEEE.
- Huynh-Thi-Le, Q., Le, T., Vo, B., & Le, B. (2015). An efficient and effective algorithm for mining top-rank-k frequent patterns. *Expert Systems with Applications*, 42(1), 156–164.
- Iqbal, S., Shahid, A., Roman, M., Khan, Z., Al-Otaibi, S., & Yu, L. (2021). Tkfm: Top-k frequent itemset mining technique based on equivalence classes. *PeerJ Computer Science*, 7, e385.
- Liu X., Gan W., Yu L., Liu Y., DP-Partfim: Frequent itemset mining using differential privacy and partition. *IEEE Transactions on Emerging Topics in Computing*, 13(3), (2024). <https://doi.org/10.1109/TETC.2024.3443060>
- Malliaridis, K., Ougiaroglou, S., Dervos, D. A., Bratsas, C., Sidiropoulos, A., & Diamantaras, K. (2024). Automatic dataset type recognition for association rule mining. In *Proceedings of the 13th hellenic conference on artificial intelligence* (pp. 1–8).
- Metwally, A., Agrawal, D., & El Abbadi, A. (2005). Efficient computation of frequent and top-k elements in data streams. In *International conference on database theory* (pp. 398–412). Springer.
- Pei, J., Han, J., Lu, H., Nishio, S., Tang, S., & Yang, D. (2007). H-mine: Fast and space-preserving frequent pattern mining in large databases. *IIE Transactions*, 39(6), 593–605.
- Pietracaprina, A., Riondato, M., Upfal, E., & Vandin, F. (2010). Mining top-k frequent itemsets through progressive sampling. *Data Mining and Knowledge Discovery*, 21, 310–326.
- Pyun, G., & Yun, U. (2014). Mining top-k frequent patterns with combination reducing techniques. *Applied Intelligence*, 41, 76–98.
- scikit-mine developers (2025). scikit-mine: Reference – datasets (synthetic data generation). <https://scikit-mine.github.io/scikit-mine/reference/datasets.html#synthetic-data-generation>. Online; accessed 4 November 2025.
- Song, W., Yang, B., & Xu, Z. (2008). Index-bittableFI: An improved algorithm for mining frequent itemsets. *Knowledge-Based Systems*, 21(6), 507–513.
- Wai-chee Fu, A., Wang-wai Kwong, R., & Tang, J. (2000). Mining n-most interesting itemsets. In *Foundations of intelligent systems: 12th international symposium, ISMIS 2000 Charlotte, NC, USA, October 11–14, 2000 proceedings 12* (pp. 59–67). Springer.
- Wan, X., & Han, X. (2024). Efficient top-k frequent itemset mining on massive data. *Data Science and Engineering*, 9(2), 177–203.
- Wang, J., Han, J., Lu, Y., & Tzvetkov, P. (2005). TFP: An efficient algorithm for mining top-k frequent closed itemsets. *IEEE Transactions on Knowledge and Data Engineering*, 17(5), 652–663.
- Yin-Ling, C., & Wai-Chee, F. A. (2004). Mining frequent itemsets without support threshold: with and without item constraints. *IEEE Transactions on Knowledge and Data Engineering*, 16(9), 1052–1069.
- Zaki, M. J. (2000). Scalable algorithms for association mining. *IEEE Transactions on Knowledge and Data Engineering*, 12(3), 372–390.
- Zaki, M. J., & Gouda, K. (2003). Fast vertical mining using diffsets. In *Proceedings of the ninth ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 326–335).
- Zaki, M. J., & Hsiao, C.-J. (2002). Charm: An efficient algorithm for closed itemset mining. In *Proceedings of the 2002 SIAM international conference on data mining* (pp. 457–473). SIAM.